



UD2.1 – Fundamentos de Kotlin

2º CFGS
Desarrollo de Aplicaciones Multiplataforma
2023-24

Importante de cara a todo el curso

Los nombres de variables, funciones, métodos, clases... deben escribirse en **inglés**.

Los nombres de los archivos también debe ser en **inglés**.

Se debe utilizar comentarios [KDoc](#) (como JavaDoc) al principio de cada archivo al menos con las etiquetas `@author` y `@versión`.

También se deben utilizar comentarios KDoc al principio de cada clase y de cada método.

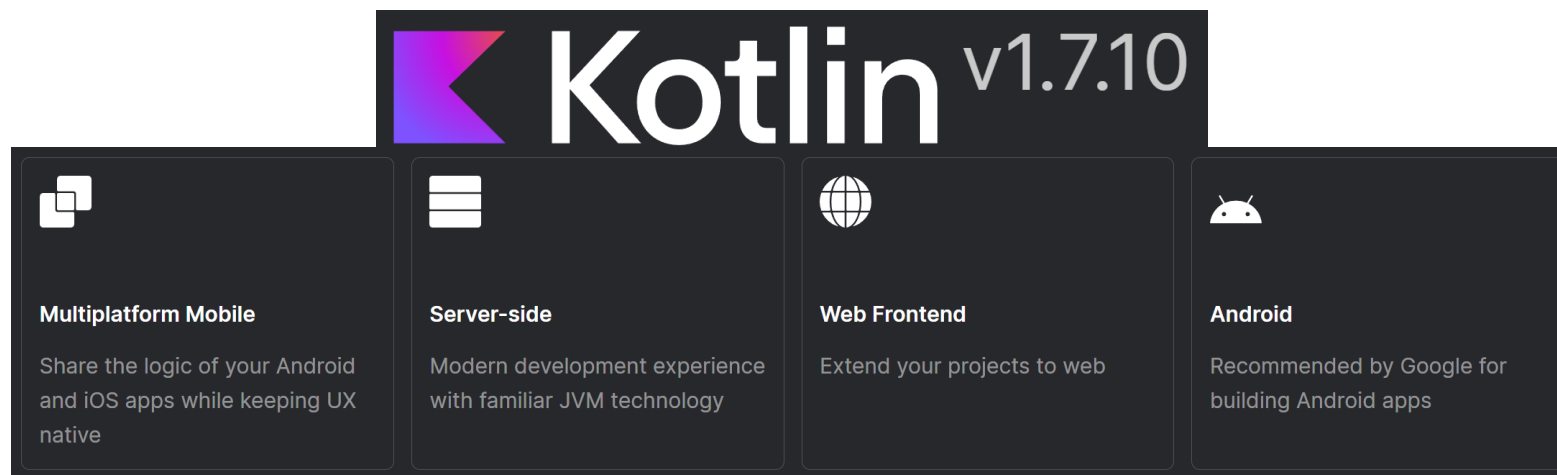
Evidentemente el código debe ser auto explicativo y en caso de haber elementos o algoritmos complejos en el código también se deben comentar.

1.- Introducción

En un principio **Java** fue el **lenguaje de programación para Android**.

En **2017** Google nombró a **Kotlin** como **lenguaje oficial para Android** equiparándolo con Java.

Kotlin tiene soporte oficial de Google y se incorpora en Android Studio.



1.- Introducción

Algunas características de Kotlin:

- Lenguaje **multiplataforma** y **multipropósito**.
- Se **compila sobre JVM** por lo que es totalmente compatible con Java y sus librerías (llamadas a Java desde Kotlin y viceversa).
- Lenguaje orientado a objetos (**POO**).
- Es un **lenguaje conciso**, evita código innecesario (hasta un 40% menos).
- Es **Null Safety**, gestiona los nulos de forma segura y evitando errores NullPointerException.
- Extraoficialmente se puede decir que es como Java al estilo de Python (sus ventajas).

1.- Introducción

En la **documentación oficial** se puede encontrar todo lo necesario sobre el lenguaje Kotlin: <https://kotlinlang.org/docs/basic-syntax.html>

También está disponible una **plataforma para aprender Kotlin** mediante ejemplos: <https://play.kotlinlang.org/byExample/overview>

1.- Introducción

En el curso se usará **Kotlin** como lenguaje de programación para Android.

Al ser Kotlin un lenguaje multipropósito se pueden desarrollar aplicaciones Android, de escritorio, web, para consola...

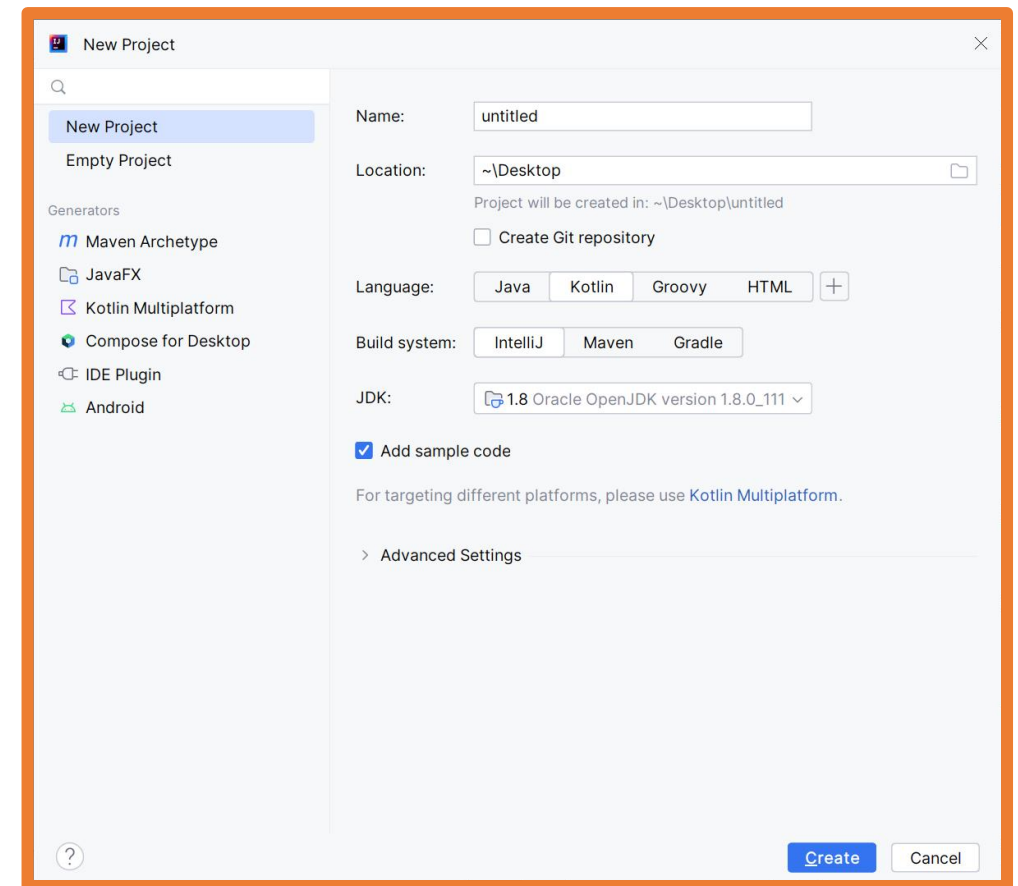
En esta unidad se verán los **fundamentos y la sintaxis de Kotlin** para más adelante **poder usar el lenguaje dentro de Android Studio** y desarrollar aplicaciones móviles Android.

Así, si se quieren crear **scripts** Kotlin y probarlos se debe tener un **JDK** de Java instalado y usar el entorno de desarrollo **IntelliJ IDEA**.

2.- Ejecutar código Kotlin

Teniendo un JDK de Java instalado en el ordenador, en **IntelliJ IDEA** se debe crear un proyecto con las siguientes opciones:

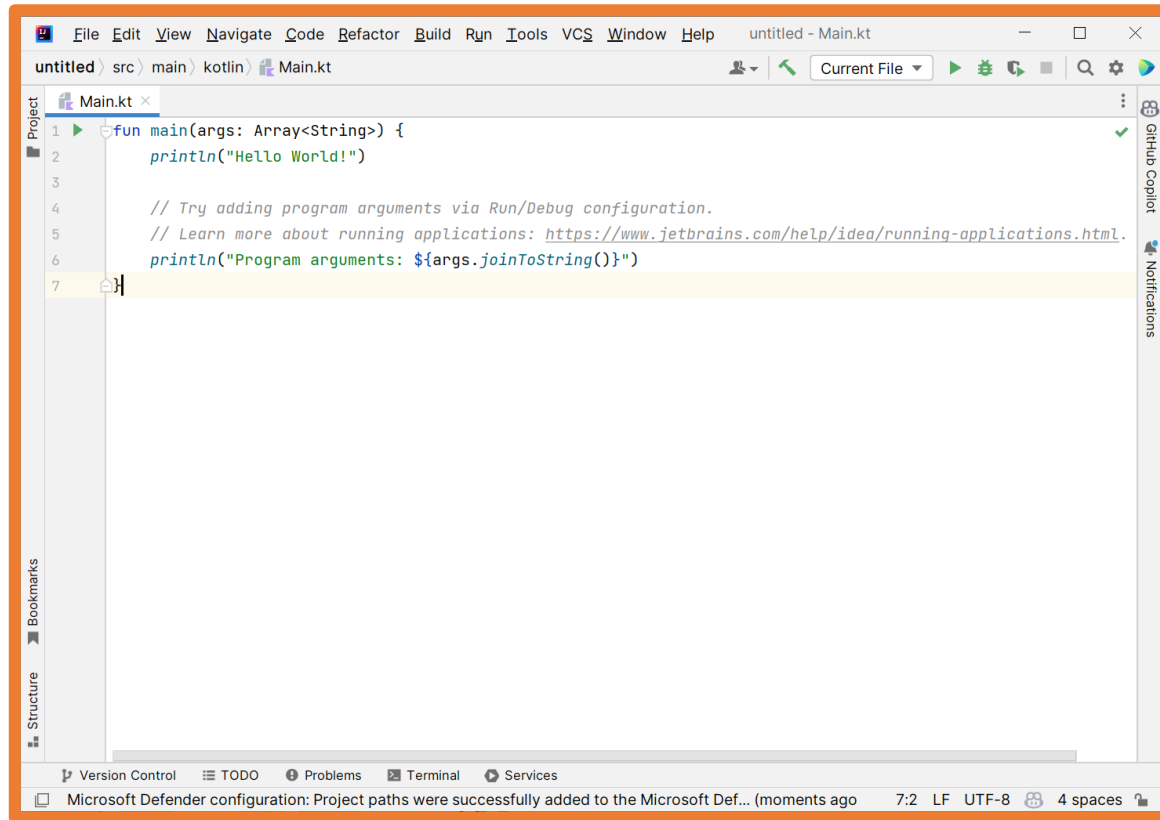
- Language: **Kotlin**
- Build system: **IntelliJ**
- **JDK**: la que se tenga instalada
- Add sample code: **marcada**



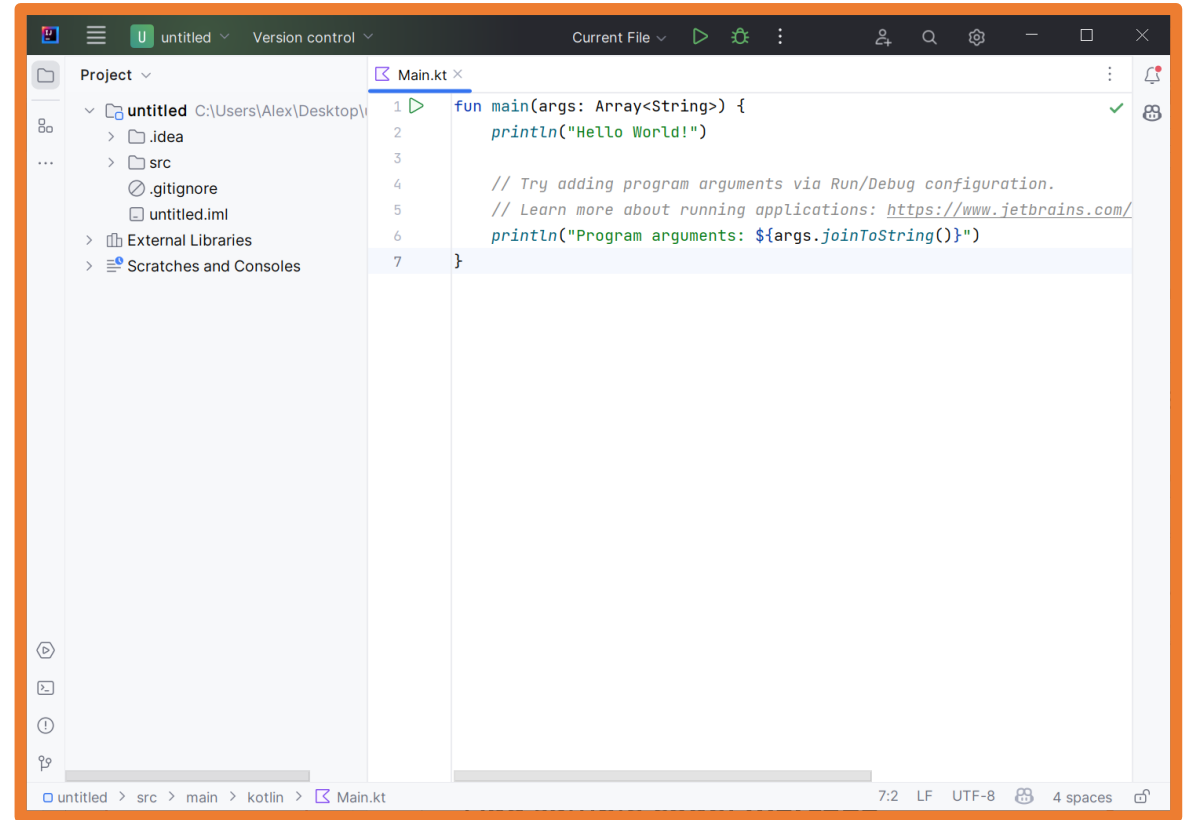
2.- Ejecutar código Kotlin

IntelliJ IDEA ha añadido una nueva interfaz, se puede usar cualquiera:

Antigua:



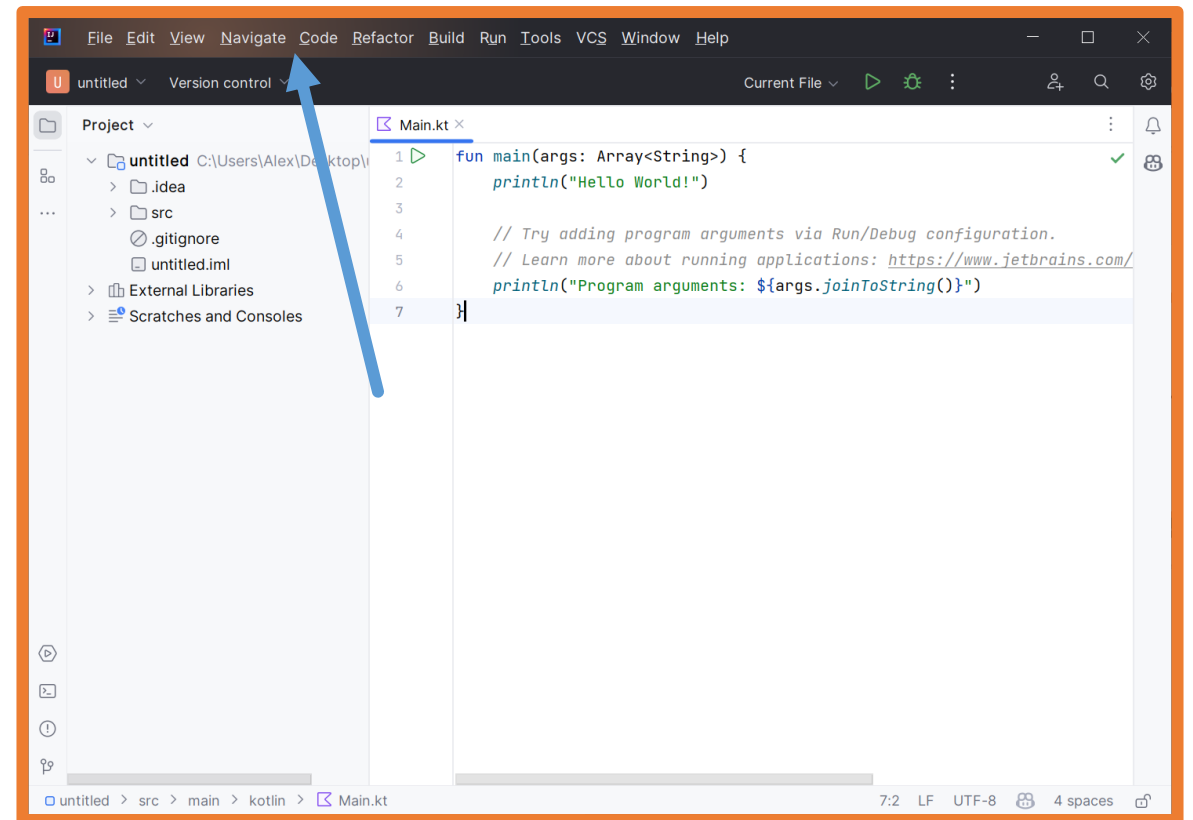
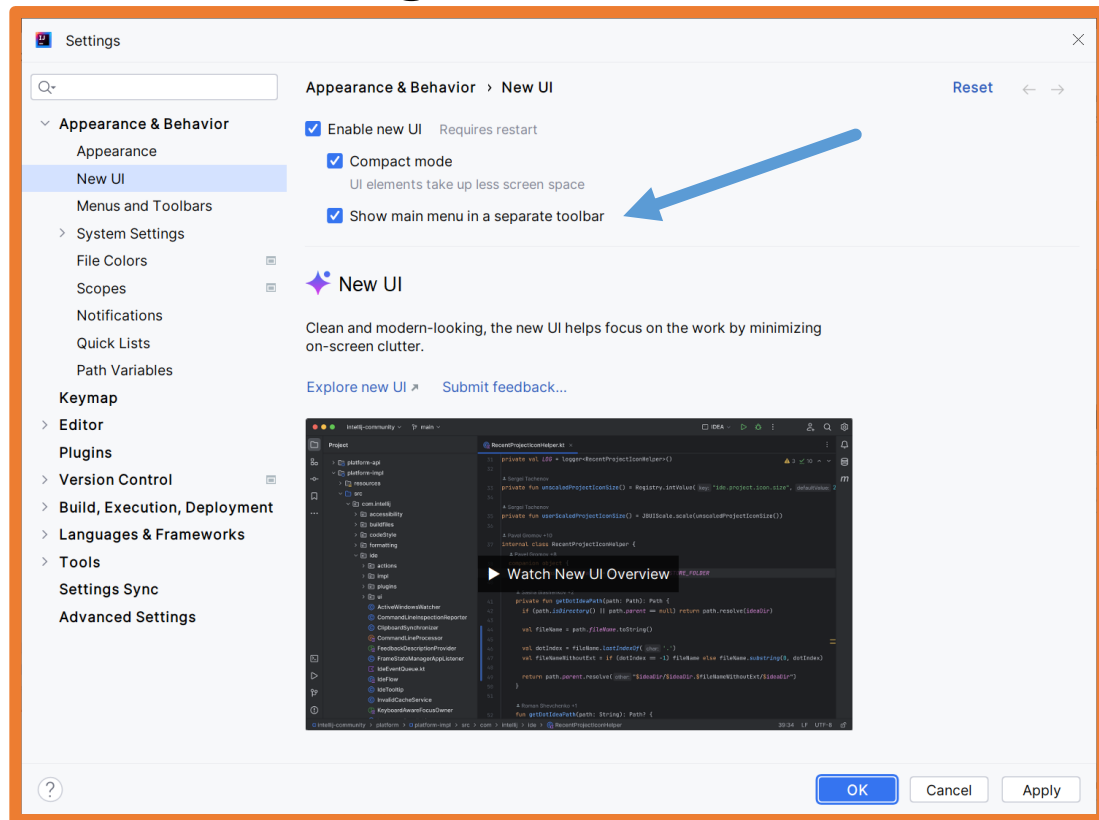
Nueva:



2.- Ejecutar código Kotlin

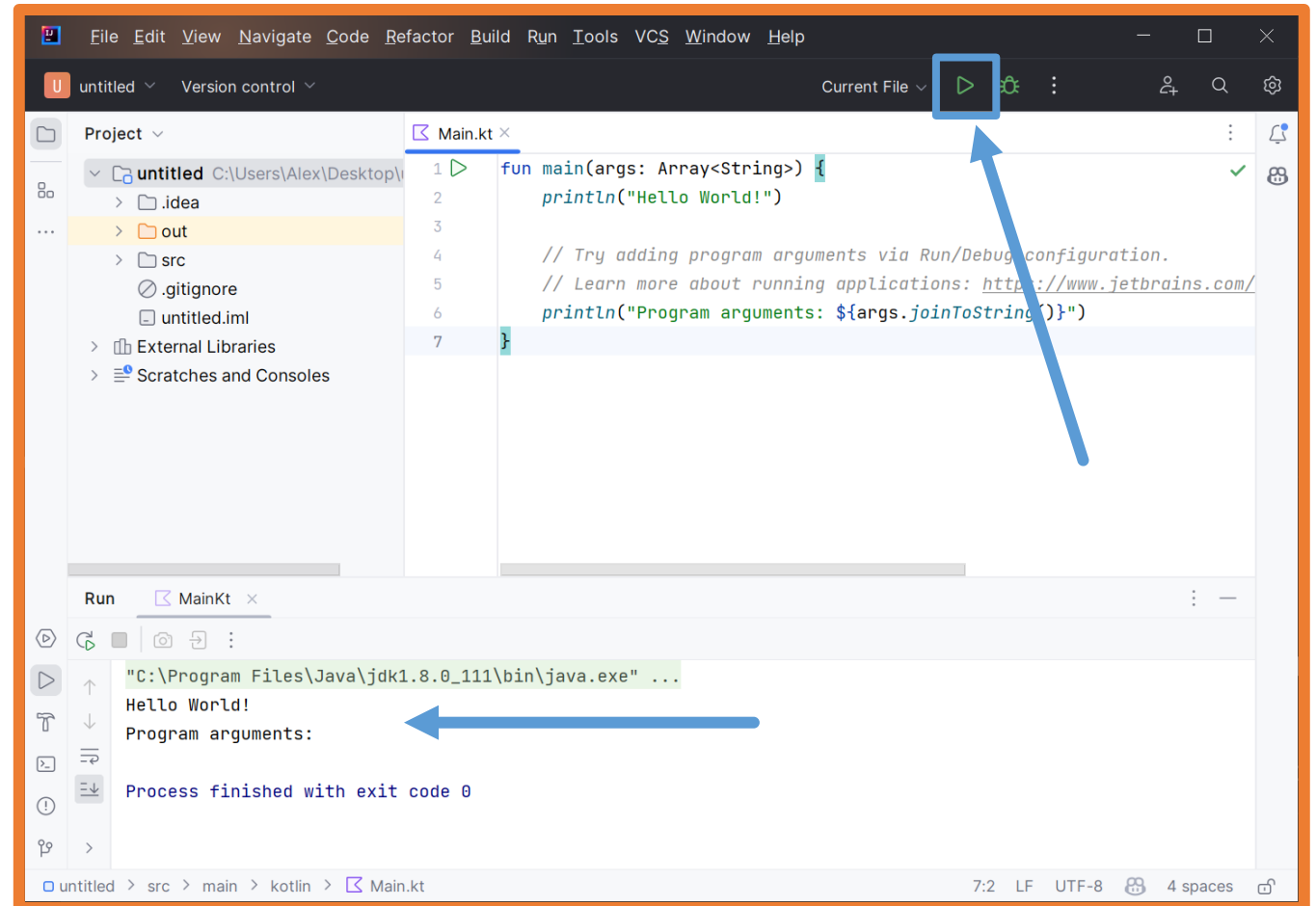
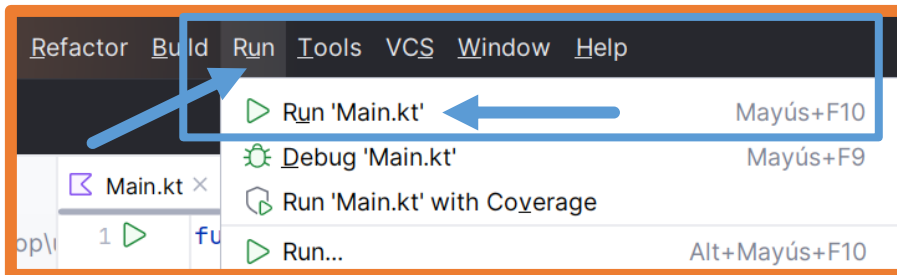
Si se quiere tener siempre visible la barra de menú en la nueva interfaz:

File → Settings



2.- Ejecutar código Kotlin

Para ejecutar un programa en IntelliJ IDEA hay varias formas de hacerlo.



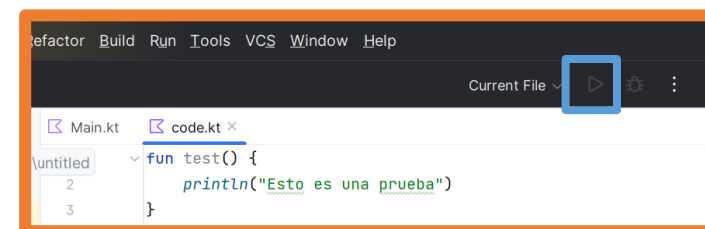
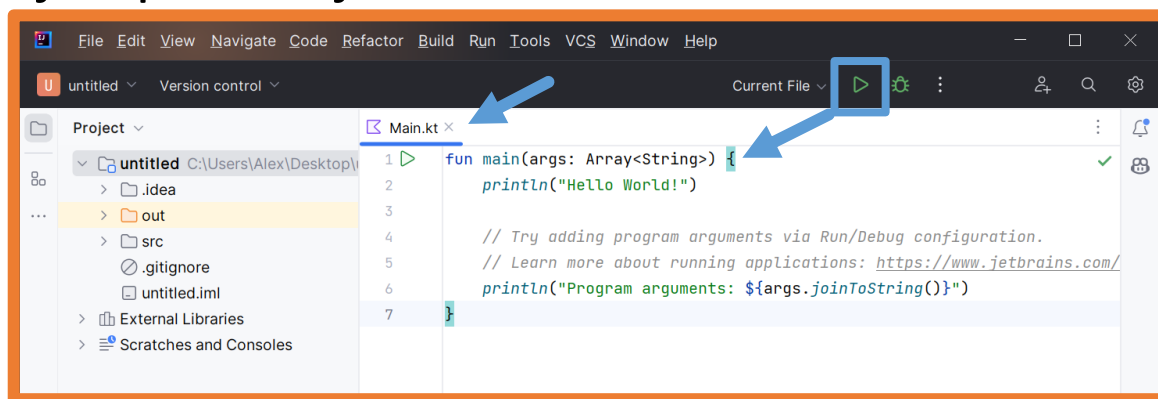
2.- Ejecutar código Kotlin

Al ejecutar un proyecto en Kotlin se ejecuta la función **main** del archivo que está activo.

Si hubiera más archivos con funciones con el nombre **main** solo se ejecutará la del archivo activo.

Si en un archivo no hay función **main** cuando esté activo el archivo no estará activo el icono **Run**.

En el ejemplo se ejecutará la función **main** del archivo **Main.kt**.



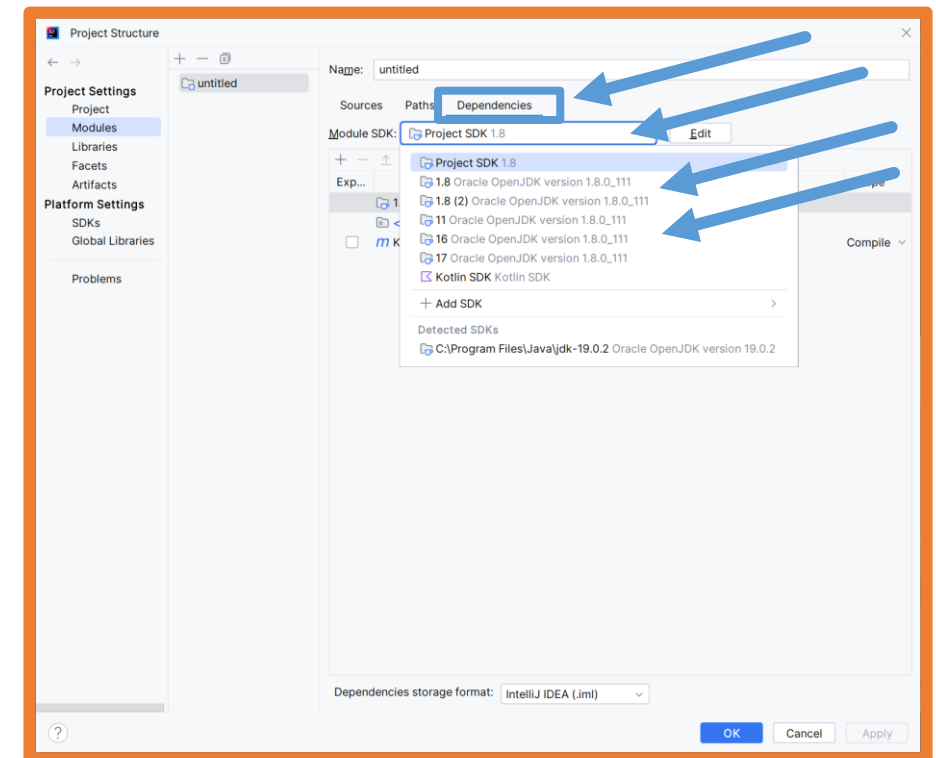
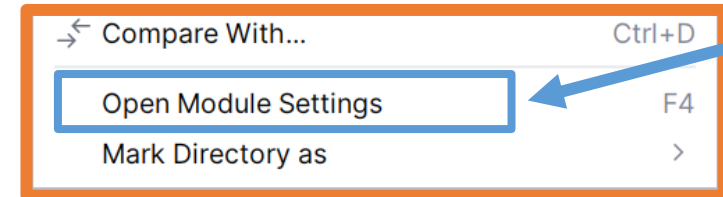
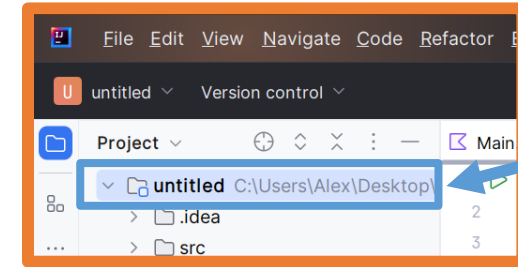
2.- Ejecutar código Kotlin

En ocasiones la opción **run** está desactivada, apareciendo el icono **play** de color gris.

Para solucionar esto se deben seguir los siguientes pasos:

- Encima de la carpeta del proyecto hacer clic derecho.
- Seleccionar la opción **Open Module Settings**.
- Seleccionar la pestaña **Dependencies**.
- En el apartado **Module SDK** cambiar la SDK seleccionada.

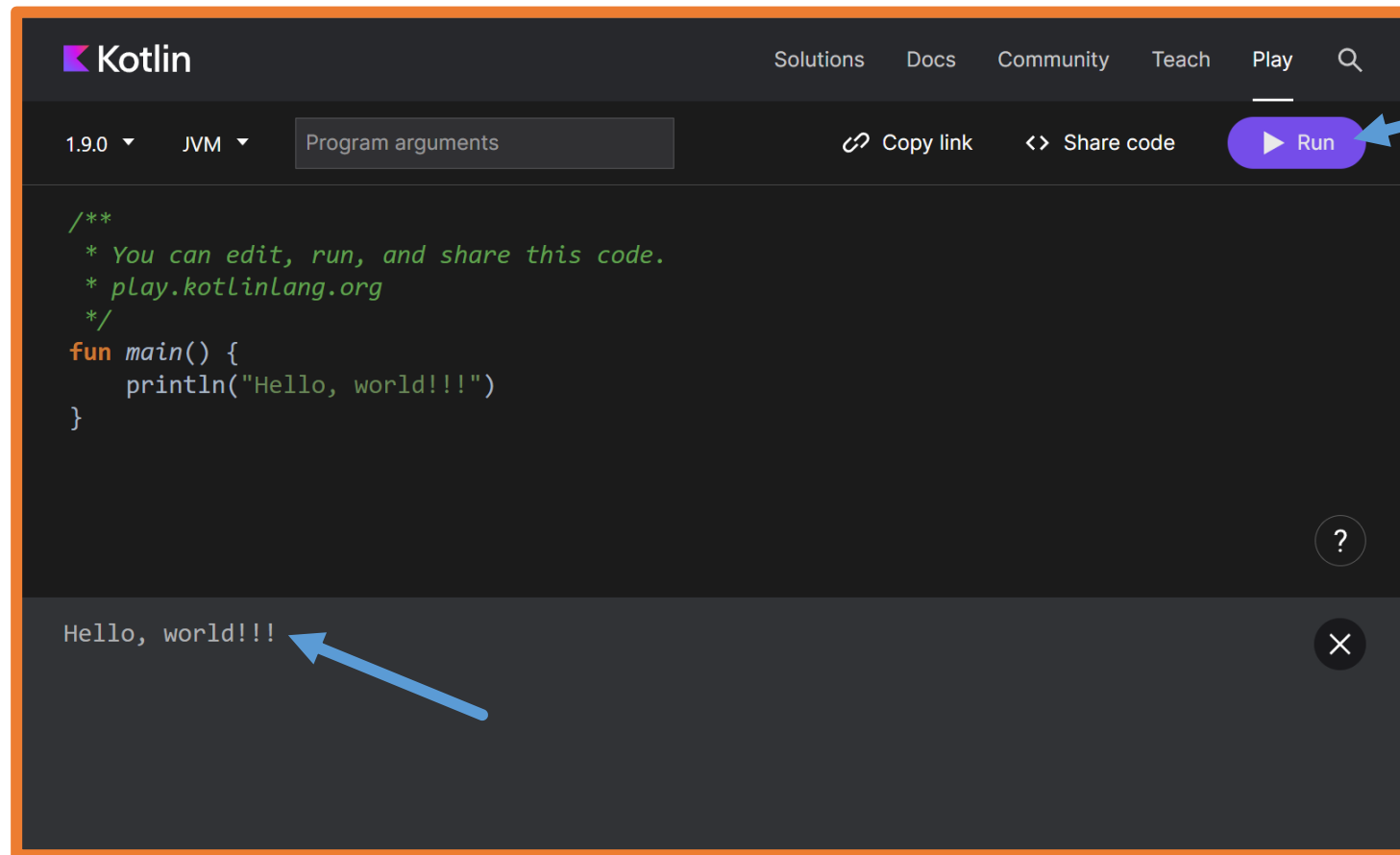
Ahora el icono **run** debería aparecer activado (verde).



2.- Ejecutar código Kotlin

También se pueden probar scripts de Kotlin en el enlace oficial:

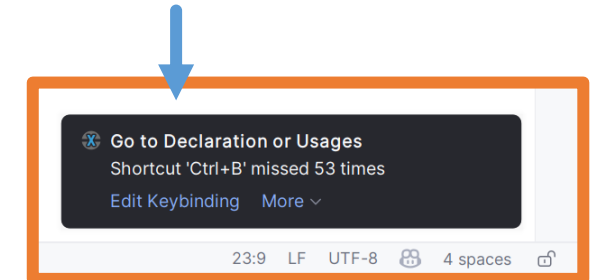
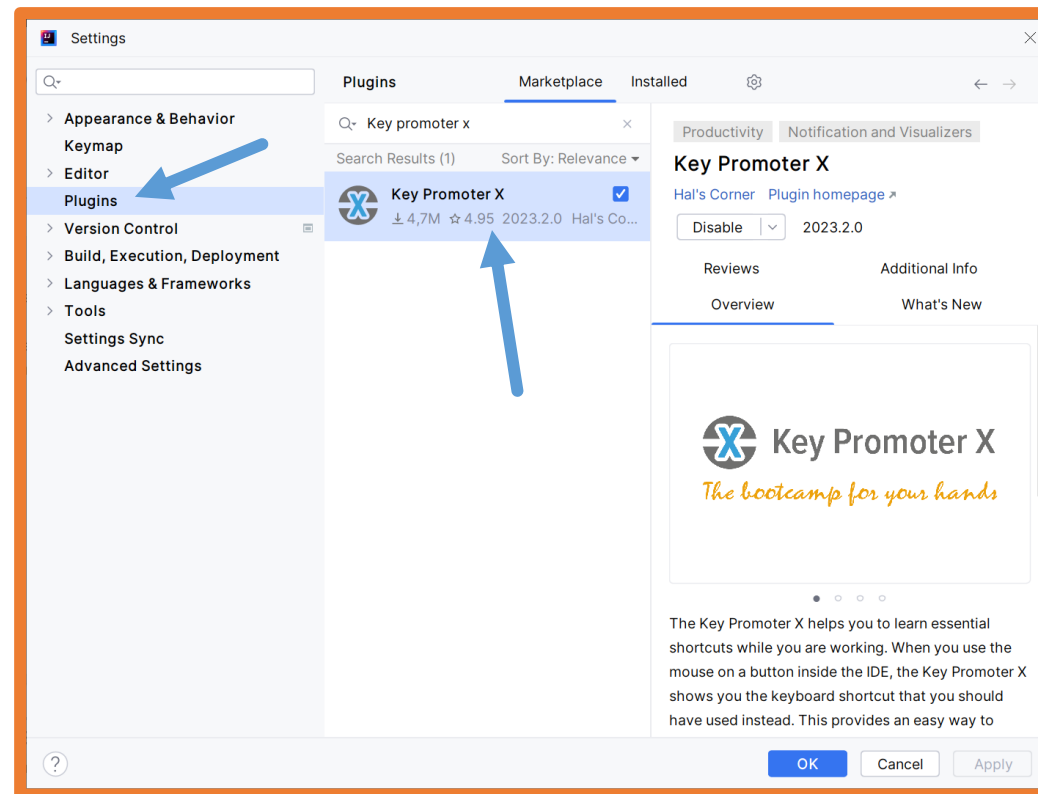
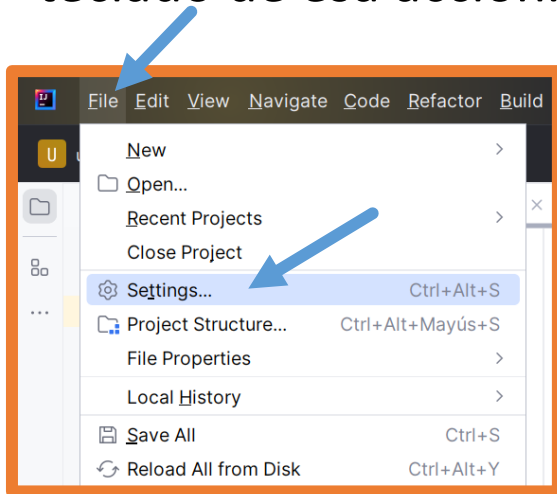
<https://play.kotlinlang.org/>



2.- Ejecutar código Kotlin

IntelliJ IDEA dispone de **plugins** para ayudar a los programadores y mejorar la productividad.

Por ejemplo, **Key Promoter X** cuando se usa el ratón muestra abajo a la derecha el atajo de teclado de esa acción.



¡Advertencia!

El curso pasado se estudió el módulo Programación.

Por esa razón **muchos conceptos se contemplarán como conocidos** en los ejemplos antes incluso de ver cómo se usan en Kotlin.

Entre ellos pueden estar:

- control de flujo (if, for...).
- funciones.
- programación orientada a objetos.
- los accesos a propiedades de objetos.
- las llamadas a funciones/métodos.
- excepciones.
- ...

3.- Guía de estilo Kotlin

En todo lenguaje de programación es importante que **el código sea legible**.

Las plataformas oficiales suelen crear [guías de estilo](#) para que los desarrolladores las sigan.

Es conveniente revisar dicha guía para desarrollar código correctamente.

También se puede usar la opción **Code → Reformat Code (Ctrl+Alt+L)** del IDE para así que se formatee el código de manera ajustada a las guías de estilo (siempre y cuando no se haya cambiado la configuración en el IDE).

4.- Saludo en Kotlin

Los archivos en Kotlin tienen la extensión **.kt**.

Al contrario que en Java **no requiere que todo sean clases**.

La función principal **main** no es necesario que se encuentre dentro de una clase.

main puede recibir argumentos pero no es necesario indicarlo.

No es necesario indicar la visibilidad public/static en la función.

La función **println** está disponible sin necesidad de usar System.out.

La función **readln** captura como cadena lo introducido en la consola por teclado.

El ; al final de cada instrucción es **opcional, por convención no debe usarse**.

Los **comentarios** tanto de línea como de bloque son como en Java: **//** y **/* */**.

Greeting.kt

```
// Programa en Kotlin  
  
fun main() {  
    println("Hello there!")  
}
```

5.- Paquetes e importaciones

Como en Java, el paquete al que pertenece el archivo y las importaciones que se realicen se colocan en la parte superior del archivo.

```
package com.alextorres.hellothere

import android.support.v7.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

6.- Tipos de datos en Kotlin

Numéricos

Se indican con las palabras reservadas **Int**, **Float**, **Long**, **Short**, **Byte** y **Double**

Decimales: **Double** (64 bits), **Float** (32 bits)

Enteros: **Long** (64 bits), **Int** (32 bits), **Short** (18 bits), **Byte** (8 bits)

Ejemplos de **literales**:

Tipo de número		Ejemplo
Entero	Int	154
	Long	154L
	Hexadecimal	0x8F
	Binario	0b00010110
Decimal	Double	154.8 1.548e2 (158x10 ²)
	Float	154.8f 154.8F

6.- Tipos de datos en Kotlin

Numéricos

Kotlin permite el uso del **carácter subrayado** (guion bajo) **en los literales numéricos** para facilitar la lectura de los mismos.

Ejemplos de **literales**:

2_000_000

44_294_051

6135_8442_0103_56100



6.- Tipos de datos en Kotlin

Caracteres

Se indican con la palabra reservada **Char**.

Se utiliza la codificación UTF-16 por lo que los literales se pueden indicar con el carácter o con su representación UNICODE entre comillas simples.

Ejemplos de **literales**:

'N' → '\u004E'
'7' → '"\u0037'

No se pueden tratar como números.

6.- Tipos de datos en Kotlin

Caracteres

Existen una serie de caracteres de escape

<code>\t</code>	Tabulación
<code>\b</code>	Retroceso
<code>\r</code>	Retorno de carro
<code>\n</code>	Salto de línea
<code>\'</code>	Apostrofe
<code>\"</code>	Comilla doble
<code>\\</code>	Backslash
<code>\\$</code>	Símbolo de dólar
<code>\u+XXXX</code>	Símbolo Unicode (4 dígitos hexadecimales)

6.- Tipos de datos en Kotlin

Booleanos

Se indican con la palabra reservada **Boolean**.

Los valores posibles son verdadero y falso y sus **literales**:

true
false

6.- Tipos de datos en Kotlin

Cadenas

Se indican con la palabra reservada **String**.

Los literales de cadenas **se indican con comillas dobles**:

"Hello there!"

"Rick Sanchez"

6.- Tipos de datos en Kotlin

Cadenas

¡IMPORTANTE!

Con las **triples comillas dobles** la cadena admite saltos de línea, se conocen como **raw strings** (cadenas en crudo) en las que cuentan todos los caracteres englobados entre las comillas.

```
"""A veces no soy yo  
Busco un disfraz mejor  
Bailando hasta el apagón  
Disculpad mi osadía"""
```

6.- Tipos de datos en Kotlin

Cadenas

¡IMPORTANTE!

Se pueden eliminar los espacios en blanco del principio y del final y los márgenes de la izquierda.

```
"" | A veces no soy yo  
  | Busco un disfraz mejor  
  | Bailando hasta el apagón  
  | Disculpad mi osadía"".trimMargin()
```

Por defecto el carácter | es el delimitador del margen pero se puede cambiar: `trimMargin(">")`

6.- Tipos de datos en Kotlin

Cadenas

¡IMPORTANTE!

Los **string templates** (plantillas de cadenas) permiten integrar las variables dentro de los strings, así usando el símbolo \$ seguido del nombre de una variable, Kotlin lo sustituirá por el valor almacenado en la variable.

Si la variable es un objeto o se quiere operar con ella se debe envolver en llaves { }.

```
"El descuento es un $discount %"
```

```
"El precio es ${product.price} €"
```

```
""" | Libreta de cuadros $notebookPrice €  
    | Bolígrafo azul $penPrice €  
    | Total: ${notebookPrice + penPrice}  
""".trimMargin()
```

6.- Tipos de datos en Kotlin

Cadenas

¡IMPORTANTE!

Se **recomienda** siempre el uso de **string templates** y de **raw strings**.

Se pueden concatenar cadenas con el carácter **+** pero se debe evitar su uso.

Se debe evitar el uso de varias instrucciones seguidas con `println`.

6.- Tipos de datos en Kotlin

Arrays

Los arrays son **estructuras de datos de longitud fija** que permiten almacenar varios valores del mismo tipo (números, caracteres, strings, booleanos, objetos).

1 2 3 4 5

"Rick", "Morty", "Summer"

7.- Declaración de variables

Mediante la palabra reservada **var** se declaran variables.

Las variables son como cajas que **almacenan valores** y estos valores pueden cambiar durante la ejecución del programa.

Kotlin es un lenguaje de **tipado estático**, esto significa que cuando una variable es de un tipo, ese tipo no puede cambiar (como Java).

Al declarar una variable se puede indicar o no el tipo de dato que se almacenara.

Si no se indica, Kotlin lo **deducirá** (inferred) dependiendo del tipo de dato que se asigne.

7.- Declaración de variables

Cuando se declara una variable obligatoriamente se debe realizar una de estas dos opciones:

- **Indicar el tipo de dato** que se almacenará.
- **Indicar el valor** que se almacenará (Kotlin deducirá el tipo de dato).

También se puede declarar una variable realizando las dos acciones:
indicar el tipo de dato y el valor que almacenará.

7.- Declaración de variables

Declaración de variables de tipos "simples"

```
var name: String = "Rick Sanchez"  
var dimension = "c-137" // el tipo de dato se deduce  
var gender: Char = 'h'  
age: Int  
age = 70  
var isAlive: Boolean = true
```


7.- Declaración de variables

Declaración de arrays

Son objetos y se crean mediante la **clase Array**, por ello disponen de los métodos get y set pero por convenciones está disponible también el uso de corchetes [].

```
// se indica el tipo de dato del array
var numbers: Array<Int> = arrayOf(1, 2, 3, 4, 5)

// se deduce el tipo de dato del array
var names = arrayOf("Rick", "Morty", "Summer")

println(numbers[1])      // numbers.get(1)
numbers[3] = 12         // numbers.set(3, 12)

println("Hay ${names.size} nombres");
```

7.- Declaración de variables

Declaración de arrays

Se puede usar el constructor de la clase Array para indicar el tamaño e incluso una **función lambda** para rellenar los elementos.

```
var array = Array( size: 10) { }           // array de 10 elementos vacíos (Unit)

var zeros = Array( size: 5) { 0 }          // array de 5 enteros todos 0

var emptys = Array( size: 5) { "" }        // array de 5 cadenas todas ""

var serie = Array( size: 100) { it }       // array de 100 enteros: 0, 1, 2 .. 99

var reverse = Array( size: 50) { 50-it }   // array de 50 enteros: 50, 49, 48 .. 0
```

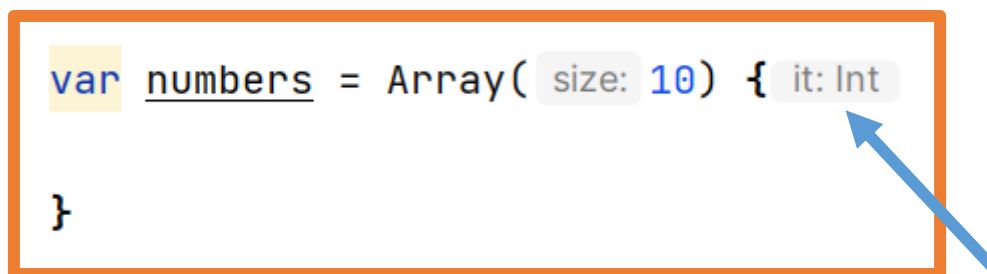
7.- Declaración de variables

Declaración de arrays

El código escrito entre llaves es una **función lambda** (anónima), que se explicarán más adelante.

En esta función lambda se dispone de la variable **it** que representa el número de iteración comenzando por cero.

Como IntelliJ IDEA por defecto tiene activadas **las pistas en el código** (hints), si se deja un salto de línea entre las llaves se mostrará esta información.



```
var numbers = Array(size: 10) {  
    it: Int  
}
```

7.- Declaración de variables

Declaración de arrays multidimensionales

Los **arrays multidimensionales** son arrays en los que cada elemento es a su vez un array.

Los más comunes son las matrices que se pueden representar como una tabla.

Hay varias maneras de declarar un array multidimensional en Kotlin:

```
// Matriz 3x4 de enteros ya inicializados
val matrix = arrayOf(
    arrayOf(5, 3, 2, 0),
    arrayOf(4, 9, 4, 5),
    arrayOf(8, 1, 7, 6)
)

// Matriz de 3x3 enteros inicializados a 0
var matrixOfInts = Array( size: 3) { Array( size: 3) { 0 } }

// Matriz de 6x5 cadenas inicializadas a ""
var matrixOfStrings = Array( size: 6) { Array( size: 5) { "" } }
```

5	3	2	0
4	9	4	5
8	1	7	6

7.- Declaración de variables

El **acceso** a los elementos de la matriz se realiza **con los índices de la fila y la columna**.

```
val matrix = arrayOf(  
    arrayOf(5, 3, 2, 0),  
    arrayOf(4, 9, 4, 5),  
    arrayOf(8, 1, 7, 6)  
)  
  
matrix[1][3] = 8  
  
for (row in matrix) {  
    for (column in row) {  
        print("$column ")  
    }  
    println()  
}
```

7.- Declaración de variables

Todos los tipos de variables son clases y en la jerarquía de Kotlin tienen una superclase que es **Any** (en Java sería Object).

Se puede declarar una variable indicando el tipo Any y luego asignar cualquier tipo de dato. Esta acción puede tener utilidad en algunos casos como se verá más adelante.

```
var number: Any = 6  
var name: Any = "Rick Sanchez"  
var isAlive: Any = true  
var numbers = arrayOf(1, 2, 3)
```

7.- Declaración de variables

Para poder usar una variable esta debe almacenar un valor.

Existe un tipo de dato que permite dejar vacía una variable **Unit** (en Java sería void).

Se puede declarar una variable indicando el tipo Any y luego asignar cualquier tipo de dato. Esta acción puede tener utilidad en algunos casos como se verá más adelante.

```
val variable1: Any
// Produce error al no haberse inicializado la variable
println("valor de variable: $variable1")

val variable2: Any = Unit
// Funciona correctamente
println("valor de variable: $variable2")
```

7.- Declaración de variables

Ámbito de las variables

El ámbito de una variable es el lugar donde se puede utilizar.

Como norma general una variable **se podrá utilizar dentro de todo el bloque de código en el que se declare que se delimita por las llaves { }.**

Una variable se podrá usar dentro de todo ese bloque, incluidos otros bloques que se contengan dentro.

Si una variable **se declara fuera de un bloque { }** se podrá utilizar **en todos los archivos del proyecto.**

8.- Conversión de tipos

Si se quiere realizar una conversión de tipo de dato siempre **se deberá usar un método de la clase del tipo de dato a convertir.**

Por ejemplo, para convertir un Double a un Int, se deberá usar sobre la variable decima el método toInt de la clase Double.

Esta acción además truncará el valor y se perderá la parte decimal.

```
var decimalNumber: Double = 12.34  
var intNumber: Int  
intNumber = decimalNumber.toInt()
```

8.- Conversión de tipos

Los métodos para la conversión de tipos suelen ir precedidos de **to**:

toDouble()

toFloat()

toLong()

toInt()

toChar()

toShort()

toByte()

toString()

Cuando se pide un dato por teclado este siempre es una cadena, así que si se pide un número se debe convertir a entero o decimal según convenga:

```
var number = readln().toInt()
```

9.- Declaración de constantes

Se usa la palabra reservada **val** para declarar constantes.

Su uso es similar al de las variables declaradas con `val` con la excepción de que una vez asignado un valor ya no se podrá cambiar durante la ejecución.

```
val age: Int = 23  
age = 24
```

Aunque sí que se pueden asignar durante la ejecución del programa por ejemplo tras pedir un dato al usuario.

```
val age: Int  
age = 24
```

```
val age: Int  
age = readln().toInt()
```

9.- Declaración de constantes

Los array y todos los objetos se pueden declarar como val y posteriormente cambiar los valores que almacenan.

Esto es debido a que son objetos y como val no se podrá cambiar la instancia que almacena, aunque sí se podrán cambiar los valores.

```
val numbers = arrayOf(1, 2, 3, 4, 0)
// Se puede cambiar el contenido de los elementos
numbers[4] = 5

// Al declararse como var no se puede modificar el objeto
var numbers = arrayOf(1, 2)
```

9.- Declaración de constantes

Se puede usar la palabra reservada **const** previamente a la declaración con **val** para declarar constantes de valores conocidos.

Estas constantes no podrán ser asignadas en tiempo de ejecución por eso deben de estar fuera de las funciones.

Las constantes con **const** sí se pueden declarar dentro de objetos declarados como **singleton** (clase con una única instancia que se verán más adelante).

```
const val PI = 3.1416

fun main() {
    println(PI)
}
```

```
fun main() {
    const val PI = 3.1416
    println(PI)
}
```

```
object Constants {
    const val PI = 3.1416
}

fun main() {
    println(Constants.PI)
}
```

10.- Operadores

Permiten crear expresiones. Son similares a los de Java

De signo: + -

Aritméticos: + - * / %

Asignación: = += -= *= /= %=

Incremento: ++ -- (puede ser pre-incremento o post-incremento)

Relacionales: == != < > <= >=

Lógicos: && || !

A nivel de bit: and or xor inv shl (desplazamiento ←) shr (desplazamiento →)

Práctica

Actividad 1

Primeros programas en Kotlin

11.- Null Safety

Uno de los errores más comunes y temidos que puede producirse cuando se programa es el odiado **NullPointerException**.

Este error ocurre cuando el programador no inicializa un objeto o por alguna razón no se ha inicializado.

Por defecto Kotlin no permite asignar **null** a las variables para evitar posibles errores.

```
var name: String = null
```

Pero Kotlin sí que tiene soporte para el valor **null** de forma nativa.

11.- Null Safety

Para **declarar variables que puedan almacenar null** en la declaración se debe usar el símbolo ? detrás del tipo de dato de la variable.

```
var name: String = "Rick Sanchez"  
name = null  
  
var nullableName: String? = "Morty Smith"  
nullableName = null
```

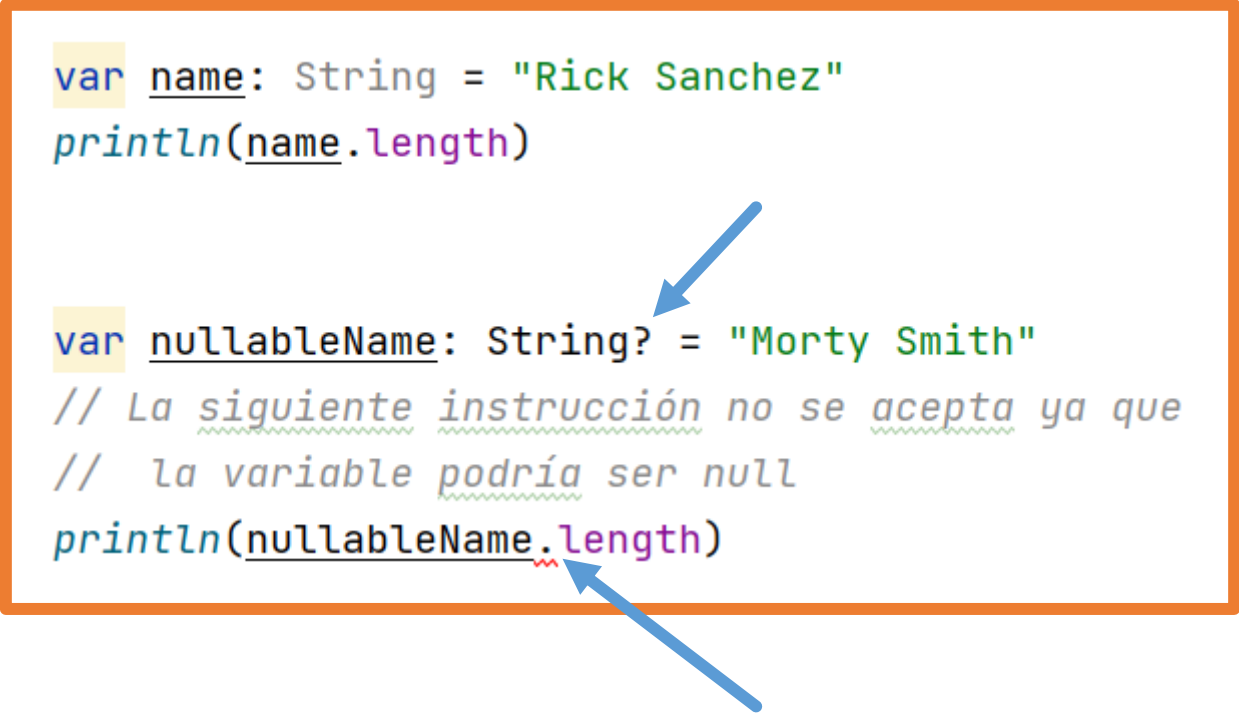


11.- Null Safety

Kotlin evitará realizar acciones que puedan dar error si la variable es null.

```
var name: String = "Rick Sanchez"
println(name.length)

var nullableName: String? = "Morty Smith"
// La siguiente instrucción no se acepta ya que
// la variable podría ser null
println(nullableName.length)
```



11.- Null Safety

Como en Java, para las variables que pueden almacenar null se puede hacer una comprobación previa antes de intentar acceder a ella.

```
var nullableName: String? = "Morty Smith"
if (nullableName != null) {
    println("Hola $nullableName!")
} else {
    println("Hola invitado!")
}
```

Con el operador ? también permite realizar las comprobaciones anteriores de una manera más sencilla y reduciendo código.

```
var nullableName: String? = "Morty Smith"
println("${nullableName?.length}")
```

Este operador permite comprobaciones simultaneas.

```
val city: String? = user?.address?.city
```

12.- Operador Elvis ?:

El operador Elvis ?: permite ofrecer un valor por defecto cuando una variable almacene el valor nulo.

```
var nullableName: String? = "Morty Smith"
println(nullableName ?: "desconocido")
nullableName = null
println(nullableName ?: "desconocido")
```



```
"C:\Program Files\Java\jdk1.8.0_111\bin\java.exe" ...
Morty Smith
desconocido

Process finished with exit code 0
```

El uso más común es conseguir un valor distinto de null cuando una propiedad es null o un método devuelve null.

```
val upperName = name?.uppercase() ?: "Desconocido"
val city = user?.address?.city ?: "Desconocida"
```

12.- Operador Elvis ?:

Este operador también se puede usar para **lanzar** una excepción.

```
var nullableName: String? = "Morty Smith"
val name = nullableName ?: throw IllegalArgumentException("El nombre es nulo")
```

Si se necesita ejecutar varias instrucciones tras el operador Elvis, estas se deberán incluir en un bloque de instrucciones tipo **run { }** (se explicará con detalle más adelante).

Se debe tener en cuenta que la última instrucción del bloque debe ser el valor a asignar en el caso de que la variable sea null.

```
val a = b ?: run {
    val valueWhenBIsNull = c.notNullValue()
    storeValue(valueWhenBIsNull)
    valueWhenBIsNull ^run // Este es el valor por defecto del operador Elvis
}
```

13.- El operador !!

Cuando se esté muy seguro de que una variable **que puede almacenar null no lo almacena**, se puede utilizar el operador **!!** para evitar la comprobación de Kotlin.

En este caso si la variable almacena null se lanzará la **excepción NullPointerException**.

```
var nullableName: String? = "Morty Smith"  
println(nullableName!!.uppercase())
```

Este operador suele aparecer en migraciones de Java a Kotlin.

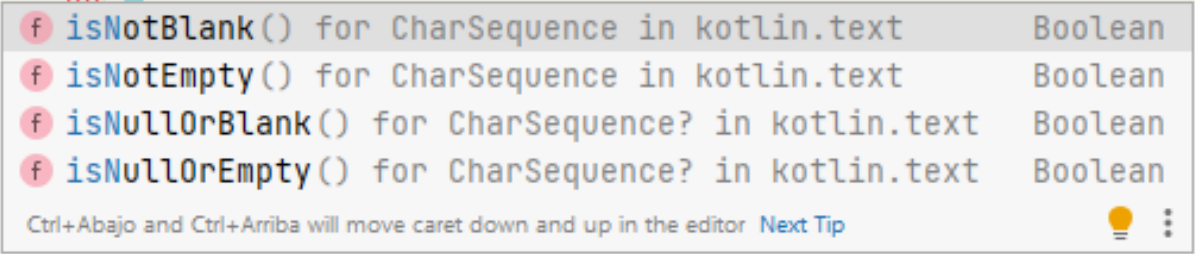
La filosofía de kotlin **es contraria a su uso**.

14.- Expresiones regulares

En todo lenguaje de programación es necesario comprobar si los valores introducidos por el usuario cumplen unas reglas específicas.

Una acción muy típica es comprobar si el usuario ha introducido un valor y si ese valor es una cadena vacía o no.

```
var name = readln()
if (name.isN)
```



f	isNotBlank()	for CharSequence in kotlin.text	Boolean
f	isEmpty()	for CharSequence in kotlin.text	Boolean
f	isNullOrBlank()	for CharSequence? in kotlin.text	Boolean
f	isNullOrEmpty()	for CharSequence? in kotlin.text	Boolean

Ctrl+Abajo and Ctrl+Arriba will move caret down and up in the editor [Next Tip](#)

Estas comprobaciones en ocasiones pueden no ser suficientes para los requisitos del programa, en ese caso se deben usar **expresiones regulares**.

14.- Expresiones regulares

Las expresiones regulares son **patrones de caracteres** que permiten **comprobar** si **una cadena de caracteres** se ajusta al patrón o no.

Ejemplos típicos de comprobaciones con expresiones regulares:

- Que un nombre no tenga números.
- Que tenga una longitud concreta.
- Que se siga un orden en los caracteres (DNI → 8 números y 1 letra).
- Que sea un email.
- ...

Las expresiones regulares existen en muchos lenguajes de programación así que es importante conocer su funcionamiento.

14.- Expresiones regulares

Para crear una expresión regular en Kotlin se utiliza la clase **Regex**:

```
val checkDNI = Regex("expresión_regular")
```

Se permite indicar [modificadores](#) a la expresión regular:

```
val check = Regex("expresión_regular", RegexOption.IGNORE_CASE)
```

El diseño de expresiones regulares es una disciplina cuyo nivel de dificultad aumenta conforme se quieren comprobar patrones más complejos.

A continuación, se verá una pequeña guía del uso de las expresiones regulares que servirá para aprender a utilizarlas de una manera básica.

14.- Expresiones regulares

Agrupación

- `[ ]` `[abc]` contiene cualquier carácter de entre los indicados.
- `[^ ]` `[^abc]` contiene cualquier carácter que no sea de los indicados.
- `[ - ]` `[0-9]` contiene cualquier carácter que se encuentre en el rango.
- `[^ - ]` `[^A-B]` contiene cualquier carácter que no esté en el rango.
- `( | )` `(x|y)` contiene uno de los caracteres (separador `|`).

Cantidad de caracteres

- `{ }` `a{3}` contiene exactamente 3 'a' seguidas.
- `{ , }` `a{3,}` contiene 3 o más 'a' seguidas.
- `{ , }` `a{3,5}` contiene 3, 4 o 5 'a' seguidas.
- `*` `a*` contiene 0 o más 'a'. Similar: `a{0, }`
- `+` `a+` contiene 1 o más 'a'. Similar: `a{1, }`
- `?` `a?` contiene 0 o 1 'a'. Similar: `a{0,1}`

14.- Expresiones regulares

Inicio – fin

- `^` `^hola` empieza con "hola".
- `$` `hola$` acaba con "hola".
- `^ $` `^hola$` exactamente "hola".

Otros

- `\\s` el carácter espacio en blanco.
- `\\S` cualquier carácter que no sea espacio en blanco.
- `\\w` cualquier carácter menos símbolos.
- `\\W` solo símbolos.
- `\\C` no es una letra.
- `\\d` un dígito. Similar: `[0-9]`
- `\\D` no es un dígito. Similar: `[^0-9]`

14.- Expresiones regulares

Ejemplos de expresiones regulares en Kotlin

```
// tiene 4 minúsculas  
"[a-z]{4}"
```

```
// tiene 8 caracteres: letras y/o números  
"[a-zA-Z0-9]{8}"
```

```
// 7 u 8 dígitos seguidos de 1 letra (DNI)  
"^\\d{7,8}\\w{1}$"
```

```
// conjunto de letras, seguidas de arroba seguida, seguidas de un  
// conjunto de letras, seguidas de un punto y seguido de 2 o 3  
// letras → patrón simple para un email: rick_sanchez@mail.com  
"^\\[a-z_\\.\\]+@[a-z]+\\.\\[a-z]{2,3}$"
```

14.- Expresiones regulares

Comprobar si una variable cumple una expresión regular

Para comprobar si una variable cumple un patrón establecido en una expresión regular se pueden utilizar tanto métodos de la clase String como métodos de la clase Regex:

```
val name = "Rick"  
// Empieza con mayúscula y le siguen al menos dos minúsculas  
val checkName = Regex( pattern: "[A-Z][a-z]{2,}$")  
// Comprobación con String  
println(name.contains(checkName))  
// Comprobación con Regex  
println(checkName.matches(name))
```

La clase Regex tiene más métodos con diferentes funcionalidades, entre ellos:

replace

find

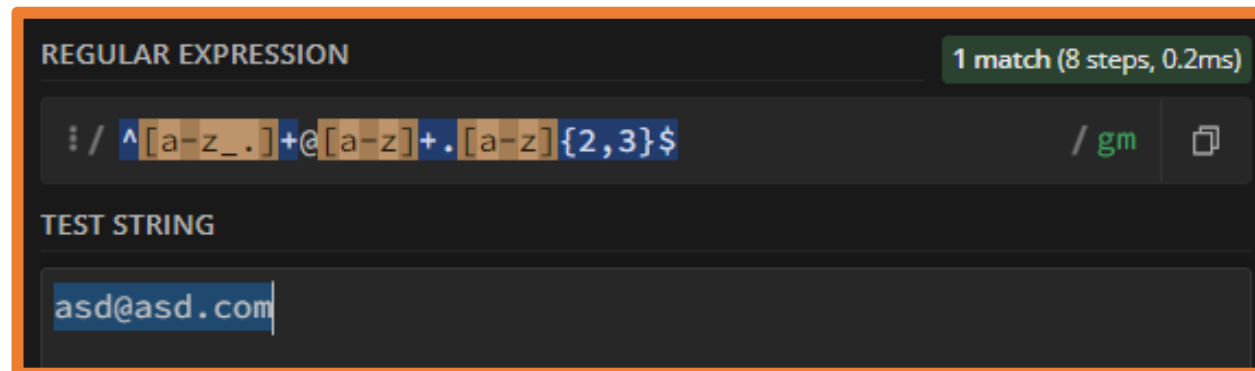
14.- Expresiones regulares

Para crear expresiones regulares se pueden consultar Cheat Sheets:

<https://quickref.me/regex>

También existen herramientas para comprobar el funcionamiento de una expresión regular:

<https://regex101.com/>



15.- Excepciones

Las excepciones se capturan con bloques **try – catch** de la misma manera que en Java.

```
try {  
    var name: Int = readln().toInt()  
} catch (e: Exception) {  
    println("Error: ${e.message}")  
}
```

Igual que en Java se pueden lanzar excepciones si se desea:

```
val number: Int  
try {  
    println("Introduce un número entre 1 y 100:")  
    number = readln().toInt()  
    if(number<1 || number>100) throw Exception("Número fuera de rango")  
} catch (e: NumberFormatException) {  
    println("Error: No se ha introducido un número")  
} catch (e: Exception) {  
    println("Error: ${e.message}")  
}
```

UD2.2 – Fundamentos de Kotlin

2º CFGS
Desarrollo de Aplicaciones Multiplataforma
2023-24

1.- Control de flujo

Instrucciones que permiten alterar el flujo normal de las instrucciones del programa.

- if
- if – else
- if – else if – else
- when
- for
- while
- do – while
- repeat

1.- Control de flujo

Guía de estilo para las llaves { }:

- Las llaves **no son necesarias** para ramas de **when** ni expresiones de **if** que **no tengan más de una rama de else** y que **quepan en una sola línea**.
- Las llaves **son necesarias** para cualquier rama de **if**, **for** o **when, do**, y para sentencias y expresiones de **while**, incluso cuando el cuerpo está vacío o contiene una sola instrucción.

2.- if, if-else y if-else if-else

Si se puede escribir en una sola línea no se ponen las llaves y es preferible escribirlo así.

```
if (number == 0) {  
    println("El número es cero")  
}
```

```
if (number == 0) println("El número es cero")
```

```
if (number1 > number2) {  
    higherNumber = number1  
} else {  
    higherNumber = number2  
}
```

```
if (number1 > number2) higherNumber = number1 else higherNumber = number2
```

```
if (number1 > number2) {  
    println("El mayor es: $number1")  
} else if (number1 < number2) {  
    println("El mayor es: $number2")  
} else {  
    println("Los dos números son: $number1")  
}
```

3.- if como una expresión

La instrucción **if** se puede utilizar como una expresión, por ejemplo para asignar un valor a una variable como en el ejemplo ya visto:

```
if (number1 > number2) higherNumber = number1 else higherNumber = number2
```

En este caso se puede escribirla expresión de la siguiente forma, siendo obligatorio incluir la rama del **else**.

```
higherNumber = if (number1 > number2) number1 else number2
```

Si se usa como una expresión **la rama de else es obligatoria.**

3.- if como una expresión

Al usar la instrucción **if** como expresión también se pueden utilizar las llaves para realizar varias instrucciones, en este caso la última instrucción de las llaves debe ser el valor a asignar.

Si se usa como expresión y además, se usan bloques de código la última expresión debe ser el valor a asignar sin necesidad de igualar a la variable.

```
var highNumber = if (number1 > number2) {  
    // instrucciones  
    number1  
} else {  
    // instrucciones  
    number2  
}  
println("El mayor es $highNumber")
```

Práctica

Actividad 2

Validando datos

4.- when

Condición con múltiples ramas (similar a switch de Java):

```
when (number) {  
  1 -> println("La variable es 1")  
  2 -> println("La variable es 2")  
  else -> println ("La variable no es ni 1 ni 2")  
}
```

Si hay varias instrucciones en alguna rama, se debe poner las llaves a todas las ramas:

```
when (number) {  
  1 -> {  
    // instrucciones  
  }  
  2 -> {  
    // instrucciones  
  }  
}
```

4.- when

Con **when** se pueden agrupar valores:

```
when (position) {  
    1, 2, 3 -> println("En el podio")  
    else -> println("Sigue intentándolo")  
}
```

También se pueden usar expresiones para evaluar:

```
var number:Int = 2  
var position:String = "5"  
when (number) {  
    parseInt(position) -> println("coincide el valor numérico")  
    else -> println("no coincide el valor numérico")  
}
```


4.- when

Se puede usar **in** o **!in** para usar **rangos** y **colecciones** para evaluar:

```
when (position) {  
    in 1 ≤ .. ≤ 10 -> println("En el top 10")  
    !in 11 ≤ .. ≤ 15 -> println("En la mitad")  
    else -> println("Muy atrás")  
}
```

```
val numbers = listOf(1, 2, 3, 4, 5)  
when (position) {  
    in numbers -> println("Top 5")  
    else -> println("Sigue intentándolo")  
}
```

Se puede evaluar si la variable es de un tipo o no con **is** o **!is**:

```
var variable:Any = "Rick"  
when (variable) {  
    is Int -> println("Es un número")  
    !is String -> print("No es una cadena")  
    else -> println("Es de otro tipo")  
}
```

4.- when

Hay casos en los que **la rama else es obligatoria**, por ejemplo si el resultado de la expresión a evaluar es un **booleano**:

```
// No compila ya que no se evalúan todas las posibilidades
when (number1 > number2) {
    true -> println("$number1 es mayor que $number2")
}
```

```
// compila porque se evalúan todas las posibilidades
when (number1 > number2) {
    true -> println("$number1 es mayor que $number2")
    else -> println("$number1 no es mayor que $number2")
}
```

```
// compila porque se evalúan todas las posibilidades
when (number1 > number2) {
    true -> println("$number1 es mayor que $number2")
    else -> println("$number1 no es mayor que $number2")
}
```

Si no se indica ninguna variable para evaluar when servirá para sustituir bloques if-else:

```
when {
    number1 > number2 -> print("El mayor es: $number1")
    number1 < number2 -> print("El mayor es: $number2")
    else -> print("Son iguales")
}
```

4.- when

Se puede usar como expresión, en ese caso la rama else es obligatoria a menos que el compilador pueda comprobar que se están cubriendo todas las opciones:

```
// se necesita el else para tener todas las posibilidades  
val result: String = when (number) {  
    1 -> "La variable es 1"  
    2 -> "La variable es 2"  
    else -> "La variable no es ni 1 ni 2"  
}
```

```
// no se necesita el else porque ya están todas las posibilidades  
var result: String = when (number1 > number2) {  
    true -> "$number1 es mayor que $number2"  
    false -> "$number1 no es mayor que $number2"  
}
```

6.- for

Los bucles for son un poco diferentes en Kotlin debido a que utilizan rangos, progresiones y colecciones para las iteraciones.

- **Rangos:** serie ascendente/descendente de Int, Long o Char.
- **Progresiones:** como un rango pero con un parámetro que indica el paso entre los elementos de la progresión.
- **Colecciones:** conjuntos de elementos como arrays.

5.- rangos y progresiones

Existen diferentes maneras de crear rangos y progresiones.

En la imagen de la izquierda se muestra cómo se crean, pero por defecto IntelliJ IDEA tiene activadas **las pistas en el código** (hints).

Una vez creado el rango se verá como en la imagen de la derecha mostrando más información.

Todos los rangos se pueden usar como progresiones con paso 1 por defecto o se pueden convertir en progresiones indicando el paso explícitamente como en el último ejemplo.

```
0.rangeTo(10)           // rango de 0 a 10 incluido
0..10                  // rango de 0 a 10 incluido

0.rangeUntil(10)       // rango de 0 a 9 incluido
0 until 10              // rango de 0 a 9 incluido
0..< 10                 // rango de 0 a 9 incluido

'a' .. 'z'             // rango de a hasta z incluida

10.downTo(0)           // Progresión de 10 a 0 incluido de 1 en 1
10 downTo 0            // Progresión de 10 a 0 incluido de 1 en 1

0..10 step 2           // Progresión de 0 a 10 incluido de 2 en 2
```

```
0.rangeTo( other: 10)
0 ≤ .. ≤ 10

0.rangeUntil( other: 10)
0 ≤ until < 10
from until from until
0 ≤ .. < 10

'a' ≤ .. ≤ 'z'

10.downTo( to: 0)
10 ≥ downTo ≥ 0

0 ≤ .. ≤ 10 step 2
```

6.- for

Los bucles for utilizan **rangos**, **progresiones** y **colecciones** y para recorrerlos se indica con la palabra **in**.

```
// si el cuerpo ocupa una sola línea mejor no poner bloque
for (i in 1 ≤ .. ≤ 3) print("$i ") // 1, 2, 3
```

```
// el cuerpo del for puede ser un bloque
for (i in 0 ≤ until < 5) {
    // 0, 1, 2, 3, 4
}
```

```
for (i in 10 ≥ downTo ≥ 0) {
    // 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0
}
```

```
for (i in 1 ≤ .. ≤ 14 step 3) {
    // 1, 4, 7, 10, 13
}
```

```
for (i in 10 ≤ until < 25 step 2) {
    // 10, 12, 14, 16, 18, 20, 22, 24
}
```

```
for (i in 10 ≥ downTo ≥ 0 step 2) {
    // 10, 8, 6, 4, 2, 0
}
```

6.- for

También funciona con rangos de caracteres

```
for (letter in 'c' ≤ .. ≤ 'j') {  
    // c d e f g h i j  
}  
  
for (letter in 'A' ≤ .. ≤ 'Z') {  
    // A B C ... X Y Z  
}  
  
//for (letter in '\u0021'..'\'u007E') {  
for (letter in '!' ≤ .. ≤ '~') {  
    // De UTF16 0021 a UTF16 007E  
}  
  
for (letter in 'c' ≤ .. ≤ 'j' step 3) {  
    // c f i  
}  
  
for (letter in 'z' ≥ downTo ≥ 'm' step 2) {  
    // < x v t r p n>  
}
```



! " # \$ % & ' () * + , - . / 0 1 2 3 4 5 6 7 8 9 : ; < = >
? @ A B C D E F G H I J K L M N O P Q R S T U V W X Y Z [\
] ^ _ ` a b c d e f g h i j k l m n o p q r s t u v w x y z
{ | } ~

6.- for

La instrucción for permite recorrer objetos que provean un **iterator** como strings, listas, arrays o colecciones.

Se puede acceder a la posición de dos maneras.

```
for (letter: Char in "Rick Sanchez") {  
  
}  
  
var letters = listOf('a', 'b', 'c', 'd')  
for (letter: Char in letters) {  
  
}  
  
for (letter in letters) { //se deduce el tipo de dato  
  
}
```

```
//var myArray = arrayOf(0, 1, 2, 3, 4, 5, 6, 7, 8, 9)  
var myArray = (0 until 9).toList()  
  
for (i in myArray.indices) {  
    println(myArray[i])  
}  
  
for ((index, value) in myArray.withIndex()) {  
    println("$index.- $value")  
}
```


7.- while y do-while

while: primero comprueba la condición, es posible que no se ejecute el cuerpo

```
while (quantity < 100) {  
    // instrucciones  
    quantity++  
}
```

do-while: la condición se evalúa después de ejecutar el cuerpo, así el cuerpo se ejecuta al menos una vez.

```
do {  
    // instrucciones  
} while (quantity-- > 0)
```

8.- repeat

La función **repeat** sirve para repetir el código del cuerpo las veces indicadas, internamente ejecuta un bucle for.

```
repeat( times: 100) { it: Int
|   println("Los 'Billetes Bart' no son de curso legal.")
| }

repeat( times: 100) { index ->
|   println("${index+1}.- Los 'Billetes Bart' no son de curso legal.")
| }
```

9.- break, continue y return

Kotlin dispone de las tres maneras típicas de romper el flujo de un bucle.

- **return**: finaliza el bucle y la función donde se encuentre el mismo.
- **break**: finaliza el bucle donde se encuentre.
- **continue**: finaliza la iteración actual del bucle donde se encuentre.

9.- break, continue y return

En Kotlin las instrucciones se pueden etiquetar mediante el carácter **@** de la siguiente manera **nombreEtiqueta@** de esta manera se puede romper el flujo de cualquier serie de bucles anidados.

De esta manera poniendo etiquetas, un **break** finalizará el bucle al que esté etiquetando.

```
firstFor@ for (i in 1 ≤ .. ≤ 10) {  
    for (j in 1 ≤ .. ≤ 5) {  
        if (i%2 == 0 && j == 2) break@firstFor  
        println("$i - $j")  
    }  
}
```

Práctica

Actividad 3

Control de flujo en Kotlin

10.- Colecciones

Las **colecciones** son un tipo de dato que permiten almacenar un número variable de elementos, cero o más.

Principalmente en una colección se almacenarán elementos del mismo tipo de dato.

Se pueden almacenar datos de diferente tipo pero en este caso se deberá tener especial cuidado cuando se use la colección.

Las colecciones pueden almacenar el valor **null** en alguno de sus elementos.

10.- Colecciones

Existen cuatro tipos de colecciones en Kotlin:

Array:

- Colección de elementos.
- Se accede a los elementos mediante el índice de su posición.
- Se pueden repetir elementos.
- Dos arrays no son iguales aunque contengan los mismos elementos en la misma posición.

List:

- Colección **ordenada** de elementos.
- Se accede a los elementos mediante el índice de su posición.
- Se pueden repetir elementos.
- Dos List son iguales si contienen los mismos elementos y en la misma posición.

Set:

- Colección de elementos **sin orden**.
- No se pueden repetir elementos dentro de un Set.
- Dos Set son iguales si contienen los mismos elementos sin importar el orden.

Map:

- También llamados diccionarios.
- Un Map es un conjunto de pares clave-valor (key-value), no se puede repetir la clave.
- Dos Map son iguales si contienen los mismos pares clave-valor sin importar el orden.

10.- Colecciones

Para las List, los Set y los Map, Kotlin dispone de dos versiones.

- **Immutable:**

- No se permite ni añadir ni eliminar elementos a la colección.

- No se permite modificar ningún elemento de la colección.

- **Mutable:**

- Se permite añadir, eliminar y modificar los elementos de la colección.

Sean mutables o no se pueden declarar tanto con **val** como con **var**.

Es su funcionamiento interno el que permite que se puedan modificar o no.

11.- List

Creación de List:

```
// Se indica el tipo de dato de los elementos
val numbers: List<String> = listOf("uno", "dos", "tres", "cuatro", "cinco")
```

```
// Se deduce el tipo de dato de los elementos, todos son iguales → String
val numbers = listOf("uno", "dos", "tres", "cuatro", "cinco")
```

```
//Se indica el tipo de dato
val list: List<Any> = listOf("Rick", 70, "c-137", 'H')
```

```
//Se deduce el tipo de dato. Diferente tipo → Any
val list = listOf("Rick", 70, "c-137", 'H')
```

Creación de MutableList:

```
// Se indica el tipo de dato pero no se añaden datos
val numbers: MutableList<Int> = mutableListOf()
```

```
// Se indica el tipo de dato de los elementos
val numbers: MutableList<Int> = mutableListOf(1, 2, 3, 4)
```

```
// Se deduce el tipo de dato de los elementos, todos son iguales → Int
val numbers = mutableListOf(1, 2, 3, 4)
```

```
//Se indica el tipo de dato
var list: MutableList<Any> = mutableListOf(1, 'a', "qwerty", true)
```

```
//Se deduce el tipo de dato. Diferente tipo → Any
var list = mutableListOf(1, 'a', "qwerty", true)
```

11.- List

Algunas operaciones sobre List y MutableList

Las operaciones que modifican solo son aplicables a MutableList

```
val list = listOf("uno", "dos", "tres", "cuatro", "cinco")
list.size // Devuelve el valor 5
list[2] // Devuelve la cadena "tres"
list.indexOf("cuatro") // Devuelve el valor 3
list.lastIndexOf(element: "dos") // Devuelve el valor 1
list.subList(1, 3) // Devuelve la lista ["dos", "tres"]

val mutableList = mutableListOf(1, 2, 6, 3, 4, 5)
mutableList.add(6) // añade el elemento 6 al final de la lista
mutableList.add(index: 2, element: 100) // añade el elemento 100 en la posición 2 desplazando el resto a la derecha
mutableList.removeAt(index: 4) // elimina el elemento de la posición 4
mutableList[1] = 99 // cambia el valor de la posición 1 por 99
mutableList.contains(3) // devuelve true/false según se encuentre o no el valor
mutableList.sort() // Ordena ascendentemente los elementos del array
mutableList.sortDescending() // Ordena descendientemente los elementos del array
mutableList.max() // Devuelve el elemento mayor
mutableList.min() // Devuelve el elemento menor
mutableList.isEmpty() // Devuelve true/false según esté vacío o no
mutableList.isNotEmpty() // Devuelve true/false según tenga elementos o no
mutableList.shuffle() // Desordena aleatoriamente los elementos del array
mutableList.shuffled() // Devuelve un array con los elementos del array desordenados
mutableList.first() // Devuelve el primer elemento del array
mutableList.last() // Devuelve el último elemento del array
mutableList.count() // Devuelve la cantidad de elementos del array
mutableList.reverse() // Da la vuelta a los elementos del array
mutableList.reversed() // Devuelve un array con los elementos del array al revés
```

12.- Set

Creación de Set:

```
val names: Set<String> = setOf("Ana", "Jorge", "Lucía")
```

```
val names = setOf("Ana", "Jorge", "Lucía")
```

```
val set: Set<Any> = setOf("Valencia", 2022, "ES", 'V')
```

```
val set = setOf("Valencia", 2022, "ES", 'V')
```

Creación de MutableSet:

```
val names: MutableSet<String> = mutableSetOf("Ana", "Jorge", "Lucía")
```

```
val names = mutableSetOf("Ana", "Jorge", "Lucía")
```

```
val set: MutableSet<Any> = mutableSetOf("Valencia", 2022, "ES", 'V')
```

```
val set = mutableSetOf("Valencia", 2022, "ES", 'V')
```

12.- Set

Algunas operaciones sobre Set y MutableSet

Las operaciones que modifican solo son aplicables a MutableSet

```
val names = mutableSetOf("Ana", "Jorge", "Lucía")
names.add("Lucas")           // Añade un elemento nuevo
names.add("Ana")             // No añade el elemento al existir ya
names.add("ana")             // Añade el elemento → "ana" != "Ana"
names.remove(element: "Jorge") // Elimina el elemento si se encuentra. Devuelve true/false
names.contains("ana")
names.min()
names.max()
names.isEmpty()
names.isNotEmpty()
names.first()
names.last()
names.reversed()
names.shuffled()
names.isEmpty()
names.isNotEmpty()
names.count()

val set1 = setOf(1, 3, 4, 8, 7, 12, 15)
val set2 = setOf(24, 7, 5, 1, 8, 6)
// Devuelve un Set con los elementos de los dos grupos sin repeticiones
val joinedGroups = set1 union set2
// Devuelve un Set con los elementos comunes a los dos grupos
val commonElements = set1 intersect set2
// Devuelve un Set con los elementos de grupo2 que se encuentren en grupo1
val differentElements = set1 subtract set2
```

13.- Map

Creación de Map:

```
val person: Map<String, String> = mapOf(  
    "name" to "Rick Sanchez",  
    "age" to "70",  
    "estado" to "vivo",  
    "dimension" to "c-137"  
)
```

```
// Se deduce el tipo de dato tanto de la clave como del valor  
val person = mapOf(  
    "name" to "Rick Sanchez",  
    "age" to "70",  
    "estado" to "vivo",  
    "dimension" to "c-137"  
)
```

```
// Map en el que los valores pueden ser de cualquier tipo  
val person: Map<String, Any> = mapOf(  
    "name" to "Rick Sanchez",  
    "age" to "70",  
    "estado" to "vivo",  
    "dimension" to "c-137"  
)
```

Creación de MutableMap:

```
val person: MutableMap<String, Any> = mutableMapOf(  
    "name" to "Rick Sanchez",  
    "age" to 70,  
    "alive" to true,  
    "dimension" to "c-137"  
)
```

13.- Map

Algunas operaciones sobre Map y MutableMap

Las operaciones que modifican solo son aplicables a MutableMap

```
val person = mutableMapOf(  
    "name" to "Rick Sanchez",  
    "age" to 70,  
    "alive" to true  
)  
person.put("dimension", "c-137") // Añade al Map el par clave-valor indicado  
person.remove(key: "agea") // Elimina y devuelve el elemento que coincide con la clave  
person.keys // Devuelve una lista con las claves  
person.values // Devuelve una lista con los valores  
person.size // Devuelve la cantidad de pares clave-valor  
person.clear() // Vacía el Map  
// Recorrer un Map o MutableMap  
for((key, value) in person) {  
    println("$key -> $value")  
}
```

14.- Operaciones sobre colecciones

Algunas operaciones sobre colecciones devuelven un valor booleano que indica si la operación se ha realizado correctamente o no.

Como por ejemplo **add** y **remove**.

Si se quieren conocer todas las operaciones disponibles para **List**, **Set** y **Map** ya sean **mutables** o **solo lectura** se debe consultar la documentación oficial.

14.- Operaciones sobre colecciones

Existen operaciones sobre las colecciones que permiten realizar operaciones a todos los elementos de la colección y devuelven una nueva colección con los resultados.

Se podría decir que "recorren" la colección.

Estas operaciones admiten una **función lambda** lo que significa que se deben usar llaves para delimitar las instrucciones. (En la declaración de arrays se vio también el uso de una función lambda, más adelante se explican por completo).

Dentro de la función lambda estará disponible **it** como el elemento actual.

14.- Operaciones sobre colecciones

forEach recorre todos los elementos de la colección pudiendo realizar acciones sobre ellos.

```
val numbers = listOf(1, 2, 3, 4, 5, 6)
numbers.forEach { it: Int
    |     print("${it*-1} ")
    | }
}
```

filter devuelve la colección con los elementos que cumplan la condición.

```
// Con Set se realiza de la misma manera
val list = listOf("Uno", "Dos", "Tres", "Cuatro", "Cinco")
val filteredList = list.filter { it: String
    |     it.length > 3
    | }
}
```

```
val map = mapOf(1 to "Uno",
                2 to "Dos",
                3 to "Tres",
                4 to "Cuatro",
                5 to "Cinco")
val filteredMap = map.filter { it: Map.Entry<Int, String>
    |     it.value.length > 3
    | }
}
```

map devuelve la colección aplicando una transformación a cada elemento.

```
// Con Set se realiza de la misma manera
val list = listOf("Uno", "Dos", "Tres", "Cuatro", "Cinco")
val mappedList = list.map { it: String
    |     it.uppercase()
    | }
// mappedList contiene la lista con los elementos en mayúsculas
```

```
val map = mapOf(1 to "Uno",
                2 to "Dos",
                3 to "Tres",
                4 to "Cuatro",
                5 to "Cinco")
val mappedMap = map.map { it: Map.Entry<Int, String>
    |     it.value.length > 3
    | }
// mappedMap: [false, false, true, true, true]
```

15.- Secuencias

Las **secuencias** permiten optimizar las operaciones sobre las colecciones.

```
val list = listOf("Uno", "Dos", "Tres", "Cuatro", "Cinco")  
val secondList = list.filter { it.length > 3 }  
                        .map { it.uppercase() }
```

En el ejemplo anterior las dos operaciones se realizan sobre todos los elementos de la lista generando listas intermedias que no desaparecen hasta acabar todas las operaciones.

Este comportamiento cuando la colección tiene muchos elementos y/o se realizan muchas operaciones sobre la colección va a penalizar en el rendimiento de la aplicación.

15.- Secuencias

Para mejorar este comportamiento se puede convertir la colección a una secuencia, realizar todas las operaciones y finalmente volver a generar la colección.

```
val list = listOf("Uno", "Dos", "Tres", "Cuatro", "Cinco")
val secondList = list.asSequence()
                        .filter { it.length > 3 }
                        .map { it.uppercase() }
                        .toList()
```

La secuencia no genera colecciones intermedias.

15.- Secuencias

Las secuencias solo generan sus elementos cuando se utilizan por ello permiten generar una cantidad de valores infinitos que posteriormente se pueden utilizar.

Se debe tener cuidado porque se podría bloquear la aplicación esperando a que la secuencia finalice.

```
// Se genera una secuencia infinita de números impares
val oddNumbers = generateSequence( seed: 1) { it + 2 }
// Se pasa por toda la secuencia y se genera una lista con los elementos menores de 200
val oddNumbersTo200 = oddNumbers.takeWhile { it < 200 }.toList()
```

16.- Funciones

Funciones incluidas en el sistema

Como en todo lenguaje de programación, Kotlin incorpora una gran cantidad de funciones ya programadas listas para usar.

Ya se han visto algunas anteriormente: `println`, `toInt`, `toString`...

Por ejemplo, para arrays se tienen disponibles entre otras

<code>min()</code>	<code>max()</code>	<code>isEmpty()</code>	<code>indexOf()</code>
<code>sort()</code>	<code>sorted()</code>	<code>reverse()</code>	<code>reversedArray()</code>

En la documentación se pueden encontrar todas.

16.- Funciones

En kotlin las funciones se declaran con la palabra **fun**.

A continuación, se muestra un ejemplo de una función y de su llamada.

```
fun isEven(number: Int): Boolean {  
    return number % 2 == 0  
}
```

```
isEven( number: 16)
```

Las funciones pueden **no recibir parámetros ni devolver valores**.

Si no se indica valor de retorno se devuelve el tipo **Unit** que es equivalente a **void** de otros lenguajes de programación. Si no se devuelve nada se puede omitir **: Unit**.

```
fun greetingPerson(name: String) {  
    println("¡Hola $name!")  
}
```

```
fun greeting() {  
    println("¡Hola!")  
}
```

```
fun greeting(): Unit {  
    println("¡Hola!")  
}
```

16.- Funciones

Los parámetros de entrada se separan por comas y se permite el uso de la llamada **trailing comma** (coma final).

```
fun areEqual(  
    num1: Int,  
    num2: Int,  
): Boolean {  
    return if (num1==num2) true else false  
}  
  
areEqual( num1: 26, num2: 123)
```

16.- Funciones

Los parámetros de entrada son inmutables, no se puede cambiar su valor desde el cuerpo de la función, es como si se declararan con **val**.

Para modificarlos se debe crear una copia.

```
fun addOne(number: Int): Int {  
    number++           // Error: no se puede cambiar el valor  
    return number  
}
```

```
var myNumber = (0 ≤ .. ≤ 100).random()  
myNumber = addOne(myNumber)
```

```
fun addOne(number: Int): Int {  
    val newNumber = number + 1  
    return newNumber  
}
```

```
var myNumber = (0 ≤ .. ≤ 100).random()  
myNumber = addOne(myNumber)
```


16.- Funciones

Si la función se puede escribir en una sola línea se **pueden quitar las llaves** y **añadir el símbolo =** antes del cuerpo.

```
fun areEqual(  
    num1: Int,  
    num2: Int,  
): Boolean {  
    return if (num1==num2) true else false  
}  
  
areEqual( num1: 26, num2: 123)
```



```
fun areEqual(  
    num1: Int,  
    num2: Int,  
): Boolean = if (num1==num2) true else false
```

```
fun greeting(nombre: String) = println("¡Hola $nombre!")  
  
fun addition(num1: Int, num2: Int): Int = num1 + num2  
  
fun isEven(num: Int): Boolean = num % 2 == 0  
  
// si no se indica el tipo de dato de retorno Kotlin lo deduce  
fun inRange(num: Int, min: Int, max: Int) = num in min ≤ .. ≤ max  
fun inRange(num: Int, range: IntRange) = num in range
```

16.- Funciones

Se puede **usar el nombre de los parámetros en las llamadas a la función** y así cambiar el orden de los parámetros.

Igual que con los intervalos IntelliJ IDEA muestra las pistas en el código (hints) para indicar el nombre del parámetro.

```
fun greeting(  
    name: String,  
    surname: String,  
): String {  
    return "Hola $name $surname"  
}
```

```
greeting("Rick", "Sanchez")  
greeting(name="Rick", "Sanchez")  
greeting("Rick", surname="Sanchez")  
greeting(name="Rick", surname="Sanchez")  
greeting(surname="Sanchez", name="Rick")
```



```
greeting( name: "Rick", surname: "Sanchez")  
greeting(name="Rick", surname: "Sanchez")  
greeting( name: "Rick", surname="Sanchez")  
greeting(name="Rick", surname="Sanchez")  
greeting(surname="Sanchez", name="Rick")
```

16.- Funciones

Se permite **definir un valor por defecto** a los parámetros de entrada y de este manera omitir esos parámetros si se quiere.

```
fun finalPrice(  
    price: Double,  
    vat: Int = 21,  
    discount: Int = 0,  
): Double {  
    var finalPrice: Double = price  
    if (discount != 0) finalPrice -= price * discount / 100  
    return finalPrice + finalPrice * vat / 100  
}
```

```
finalPrice( price: 355.0, vat: 10, discount: 5)  
finalPrice( price: 355.0, vat: 10)  
finalPrice( price: 355.0)
```

16.- Funciones

Si se usa **valor por defecto** en la definición de la función y **nombres de parámetros en la llamada a la función**, se pueden omitir parámetros intermedios en la llamada.

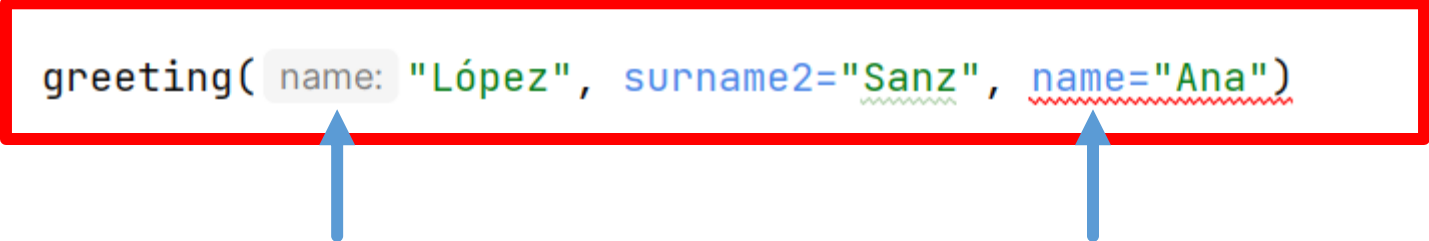
```
fun finalPrice(  
    price: Double,  
    vat: Int = 21,  
    discount: Int = 0,  
): Double {  
    var finalPrice: Double = price  
    if (discount != 0) finalPrice -= price * discount / 100  
    return finalPrice + finalPrice * vat / 100  
}  
  
finalPrice( price: 355.0, vat: 10, discount: 5)  
finalPrice( price: 355.0, vat: 10)           // descuento → valor por defecto  
finalPrice( price: 355.0, discount = 20)    // iva → valor por defecto
```

16.- Funciones

Si se usan parámetros con y sin nombre en la llamada, **los argumentos con nombre deben de ser los últimos** a no ser que ocupen su lugar.

```
greeting( name: "Ana", surname1: "López", surname2: "Sanz")  
greeting(name="Ana", surname1: "López", surname2: "Sanz")  
greeting( name: "Ana", surname2="Sanz", surname1="López")  
greeting(surname2="Sanz", name="Ana", surname1="López")
```

```
greeting( name: "López", surname2="Sanz", name="Ana")
```



16.- Funciones

Es posible **declarar un número variable de parámetros**.

Para ello se utiliza un parámetro con la palabra **vararg**.

Habitualmente suele ser el último parámetro. Si no es el último, los siguientes tendrán que pasarse usando su nombre.

Con un **intArray** o una **colección que se pueda convertir a intArray** se puede usar ***** (operador spread) para pasarlo completamente como parámetro **vararg**.

```
fun max(vararg numbers: Int): Int {  
    var max: Int = numbers[0]  
    for (number in numbers) {  
        if (number > max) max = number  
    }  
    return max  
}
```

```
max( ...numbers: 5, 32, 14, 6, 57, 7, 26, 12)  
val numbers = intArrayOf(5, 32, 14, 6, 57, 7, 26, 12)  
max(*numbers)  
max( ...numbers: 3, 67, 23, *numbers)  
max( ...numbers: 3, 67, 23, *numbers, 42, 17, 92)  
val otherNumbers: Array<Int> = arrayOf(34, 24, 15)  
max( ...numbers: 3, 67, 23, *numbers, 42, 17, 92, *otherNumbers.toIntArray())
```

16.- Funciones

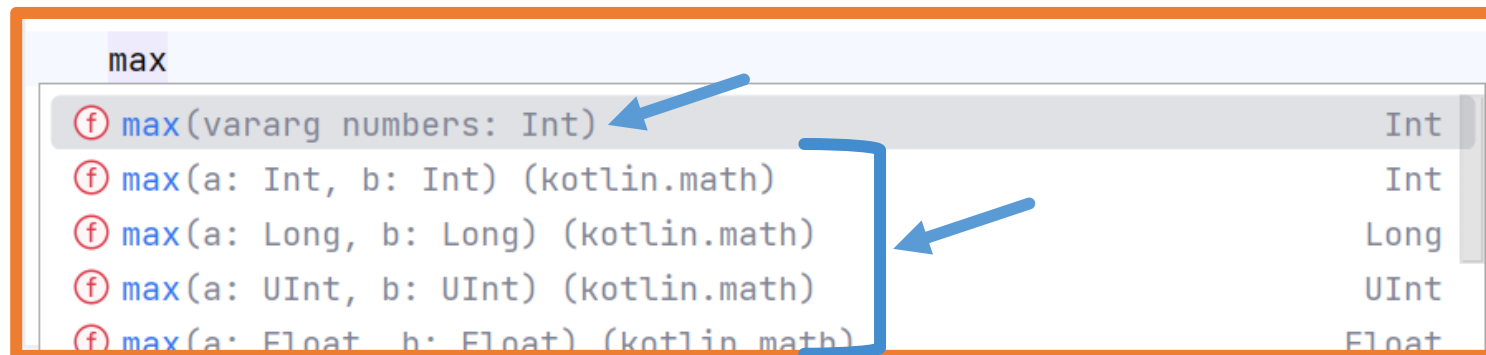
Kotlin permite la **sobrecarga** de funciones aunque estas no estén definidas en una clase.

```
fun addition() = println("No puedo sumar sin valores")

fun addition(num1: Int, num2: Int) = println("$num1 + $num2 = ${num1 + num2}")

fun addition(num1: Double, num2: Double) = println("$num1 + $num2 = ${num1 + num2}")
```

En ejemplos anteriores se ha creado la función **max** que como ya existe se realiza sobrecarga, pudiendo ver la función propia y las del sistema.



17.- Ámbito de las funciones

Igual que con las variables el ámbito de una función indica dónde se puede utilizar la función

En Kotlin existen los siguientes tipos de funciones:

- **Nivel superior** (top level)
- **Local**
- **Miembro**
- **De extensión**

Según el tipo se podrá usar en una parte de código u otra.

17.- Ámbito de las funciones

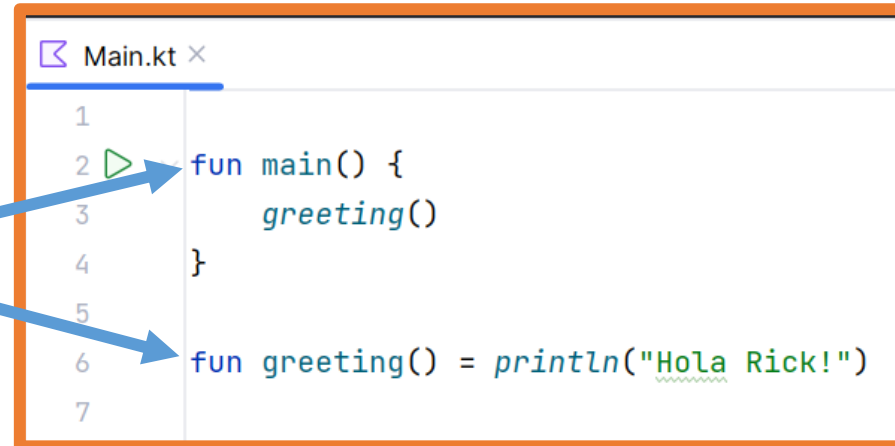
Nivel superior (top level)

se definen sin estar dentro de ningún bloque { } o clase.

Su uso suele ser como biblioteca de funciones, en este caso en un archivo se incluyen varias funciones de nivel superior que se podrán utilizar en cualquier parte.

Se podrán usar en todo el archivo propio y en el que se importe.

Top Level

A screenshot of a code editor window titled 'Main.kt'. The code contains two top-level functions: 'fun main() { greeting() }' and 'fun greeting() = println("Hola Rick!")'. Two blue arrows originate from the 'Top Level' text on the left and point to the 'fun main()' and 'fun greeting()' definitions. The code is as follows:

```
1  
2 fun main() {  
3     greeting()  
4 }  
5  
6 fun greeting() = println("Hola Rick!")  
7
```

17.- Ámbito de las funciones

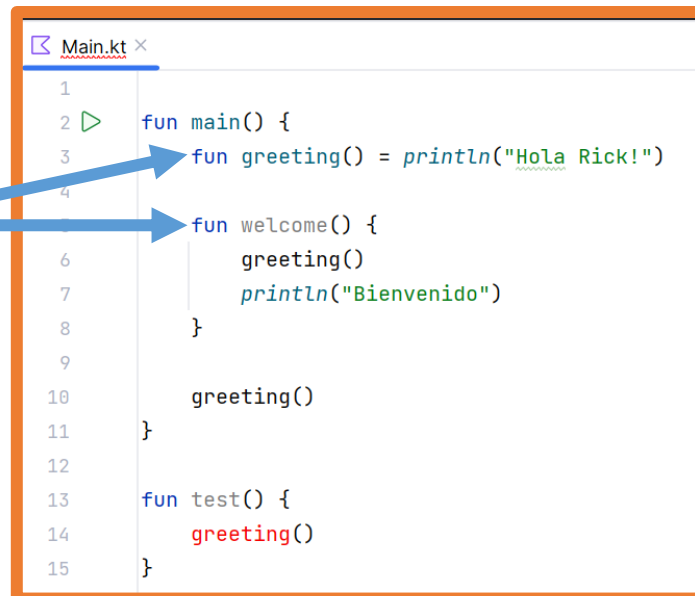
Local

Se definen dentro de otra función y se podrá usar en todo el bloque { } de esa función, incluso desde otras funciones locales de la misma función padre.

Debe estar definida antes de cualquier llamada a ella.

Pueden usar las variables locales de la función padre.

Local



```
1  
2 fun main() {  
3     fun greeting() = println("Hola Rick!")  
4  
5     fun welcome() {  
6         greeting()  
7         println("Bienvenido")  
8     }  
9  
10    greeting()  
11 }  
12  
13 fun test() {  
14     greeting()  
15 }
```

17.- Ámbito de las funciones

Miembro

Son las funciones que se definen dentro de las clases u objetos.

Son lo que en la programación orientada a objetos se llaman **métodos**.

Se estudian en el punto siguiente **POO** (Programación Orientada a Objetos).

17.- Ámbito de las funciones

De extensión

Kotlin permite **extender la funcionalidad** de las **clases** añadiendo funciones a ellas.

Estas funciones solo existen en el ámbito en el que se definen.

```
fun main() {  
    val number = (0 ≤ .. ≤ 100).random()  
    if(number.isOdd()) println("$number es impar")  
    else println("$number es par")  
}  
  
// Se añade una función a la clase Int  
fun Int.isOdd(): Boolean {  
    return this % 2 != 0  
}
```

Práctica

Actividad 4

Técnicas para reutilizar código

Actividad 5

Arrays en Kotlin



UD2.3 – Fundamentos de Kotlin

2º CFGS
Desarrollo de Aplicaciones Multiplataforma
2023-24

1.- Programación Orientada a Objetos

Como se indicó al principio de la unidad, Kotlin busca ser conciso y evitar escribir demasiado código.

Esta característica es muy visible cuando se desarrollan clases.

Se va a estudiar tanto la manera más tradicional de codificar clases como en Java y la manera aconsejada por Kotlin.

1.- Programación Orientada a Objetos

Para crear clases se usa la palabra reservada **class** como en Java.

Kotlin permite que en un mismo archivo **.kt** se definan más de una clase públicas (esto no se permite en Java).

Si en un archivo **.kt** solo se define una clase, entonces el nombre del archivo debe ser el nombre de la clase.

Si en un archivo **.kt** se definen varias clases, entonces se debe buscar un nombre representativo.

1.- Programación Orientada a Objetos

En Kotlin todas las clases hereda de la clase **Any** (en Java es Object).

La clase **Any** define tres métodos que heredarán sus hijas:

equals → indica si un objeto es igual a otro

hashCode → devuelve el código hash de un objeto

toString → devuelve la representación en String de un objeto

1.- Programación Orientada a Objetos

Ejemplo de definición de clase en Kotlin:

```
class Person {  
}
```

Kotlin es tan conciso que una clase sin cuerpo se puede definir así:

```
class Person
```

Aun sin cuerpo se puede instanciar un objeto de esta clase:

```
val morty = Person()
```

1.- Programación Orientada a Objetos

Se puede observar que la estructura de definición de una clase es similar a la usada en Java.

Los **getters** y **setters** se deben definir tras cada propiedad.

El constructor se define con la palabra **constructor**.

Su uso también es similar a Java:

```
val ps5 = Product( name: "PS5", price: 499.99)
ps5.name = "Play Statation 5"
println(ps5.name)
```

```
class Product {
    var name: String
        get() {
            return field
        }
        set(name: String) {
            field = name
        }
    var price: Double
        get() {
            return field
        }
        set(price: Double) {
            field = price
        }

    constructor(name: String, price: Double) {
        this.name = name
        this.price = price
    }
}
```

1.- Programación Orientada a Objetos

Si no se indica lo contrario las clases, propiedades y métodos **por defecto son públicas** (public).

Existen los modificadores de visibilidad:

public

private

protected

internal → visible en el mismo módulo (paquete)

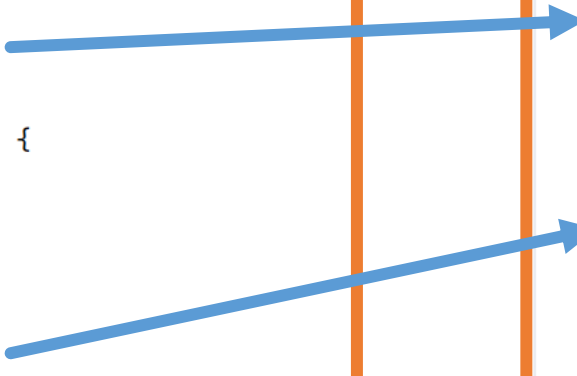
```
class Product {  
    var name: String  
        get() {  
            return field  
        }  
        set(name: String) {  
            field = name  
        }  
    var price: Double  
        get() {  
            return field  
        }  
        set(price: Double) {  
            field = price  
        }  
  
    constructor(name: String, price: Double) {  
        this.name = name  
        this.price = price  
    }  
}
```

1.- Programación Orientada a Objetos

Kotlin permite ahorrar código usando funciones de expresión (en una línea):

```
class Product {  
    var name: String  
    get() {  
        return field  
    }  
    set(name: String) {  
        field = name  
    }  
    var price: Double  
    get() {  
        return field  
    }  
    set(price: Double) {  
        field = price  
    }  
  
    constructor(name: String, price: Double) {  
        this.name = name  
        this.price = price  
    }  
}
```

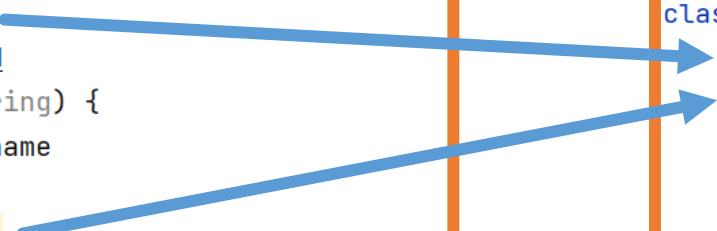
```
class Product {  
    var name: String  
    get() = field  
    set(name: String) {  
        field = name  
    }  
    var price: Double  
    get() = field  
    set(price: Double) {  
        field = price  
    }  
  
    constructor(name: String, price: Double) {  
        this.name = name  
        this.price = price  
    }  
}
```

Two blue arrows indicate the transformation of the code. The first arrow points from the 'return field' statement in the 'get()' method of the 'name' property in the left code block to the 'get() = field' expression in the right code block. The second arrow points from the 'return field' statement in the 'get()' method of the 'price' property in the left code block to the 'get() = field' expression in the right code block.

1.- Programación Orientada a Objetos

Kotlin permite ahorrar código ya que los getters y setters son implícitos:

```
class Product {  
    var name: String  
        get() = field  
        set(name: String) {  
            field = name  
        }  
    var price: Double  
        get() = field  
        set(price: Double) {  
            field = price  
        }  
  
    constructor(name: String, price: Double) {  
        this.name = name  
        this.price = price  
    }  
}
```



```
class Product {  
    var name: String  
    var price: Double  
  
    constructor(name: String, price: Double) {  
        this.name = name  
        this.price = price  
    }  
}
```

1.- Programación Orientada a Objetos

Kotlin permite ahorrar código incorporando las propiedades en la definición de la clase haciendo que el **constructor sea implícito**:

```
class Product {  
    var name: String  
    var price: Double  
  
    constructor(name: String, price: Double) {  
        this.name = name  
        this.price = price  
    }  
}
```



```
class Product(var name: String, var price: Double)
```

1.- Programación Orientada a Objetos

Kotlin permite ahorrar código:

```
class Product {  
    var name: String  
    get() {  
        return field  
    }  
    set(name: String) {  
        field = name  
    }  
    var price: Double  
    get() {  
        return field  
    }  
    set(price: Double) {  
        field = price  
    }  
  
    constructor(name: String, price: Double) {  
        this.name = name  
        this.price = price  
    }  
}
```



```
class Product(var name: String, var price: Double)
```


1.- Programación Orientada a Objetos

Con esta estructura se tiene el constructor, los getters y los setters

```
class Product(var name: String, var price: Double)
```

Si no se van a realizar acciones específicas en el constructor, los getters o los setters, esta sintaxis permite ahorrar mucho código.

Utilizando esta sintaxis este constructor se denomina **constructor primario**.

1.- Programación Orientada a Objetos

Los constructores que se incluyen en el cuerpo de la clase se denominan **constructores secundarios**.

Los constructores se declaran con la palabra reservada **constructor**.

Existe la sobrecarga de métodos como se puede ver con el constructor.

Si una propiedad no aparece en algún constructor deberá tener un valor asignado por defecto.

```
val game = Product(name: "GTA VI", price: 70.0)
val controller = Product(name: "Dual Sense")
```

```
class Product {
    var name: String
    var price: Double = -1.0

    constructor(name: String, price: Double) {
        this.name = name
        this.price = price
    }

    constructor(name: String) {
        this.name = name
    }

    // constructor(name: String) : this(name, -1.0)
}
```

```
class Product(var name: String, var price: Double) {
    constructor(name: String) : this(name, price: -1.0)
}
```

```
class Product(var name: String) {
    var price: Double = -1.0

    constructor(name: String, price: Double) : this(name) {
        this.price = price
    }
}
```

1.- Programación Orientada a Objetos

Si se usan combinados el constructor primario con constructores secundarios, se debe usar la palabra **this** en los constructores secundarios.

Mediante **this** se llama al constructor primario (equivalente a super en Java).

```
class Product {  
    var name: String  
    var price: Double = -1.0  
  
    constructor(name: String, price: Double) {  
        this.name = name  
        this.price = price  
    }  
  
    constructor(name: String) {  
        this.name = name  
    }  
  
    //    constructor(name: String) : this(name, -1.0)  
}
```

```
class Product(var name: String, var price: Double) {  
    constructor(name: String) : this(name, price: -1.0)  
}
```

```
class Product(var name: String) {  
    var price: Double = -1.0  
  
    constructor(name: String, price: Double) : this(name) {  
        this.price = price  
    }  
}
```

1.- Programación Orientada a Objetos

Puede no haber constructores.

En ese caso en la declaración de las propiedades se debe asignar un valor.

```
class Product {  
    var name: String = ""  
    var price: Double = -1.0  
}
```

```
val game = Product()  
game.name = "GTA VI"  
println("Se vende ${game.name}")
```

1.- Programación Orientada a Objetos

Si se usa el constructor primario pero se requiere realizar acciones sobre las propiedades se debe usar un bloque de instrucciones **init**.

```
class Product(var name: String, var price: Double) {  
    init {  
        name = name.uppercase()  
    }  
}
```

```
val product = Product( name: "Nintendo Switch", price: 350.0)  
println("Se vende ${product.name}")  
// Mostrará: Se vende NINTENDO SWITH
```

1.- Programación Orientada a Objetos

Se pueden definir funciones miembro (métodos) en las clases.

```
class Product(var name: String, var price: Double) {  
    fun finalPrice(percentage: Double) = price - price * percentage / 100  
}
```

La misma función definida de manera extendida:

```
class Product(var name: String, var price: Double) {  
    fun finalPrice(percentage: Double): Double {  
        return price - price * percentage / 100  
    }  
}
```

1.- Programación Orientada a Objetos

Si se sobrescribe una función heredada (ya existe en la clase madre), se debe añadir la palabra **override** en la definición de la función.

```
class Product(var name: String, var price: Double) {  
    override fun toString(): String {  
        return "$name: $price €"  
    }  
}
```

1.- Programación Orientada a Objetos

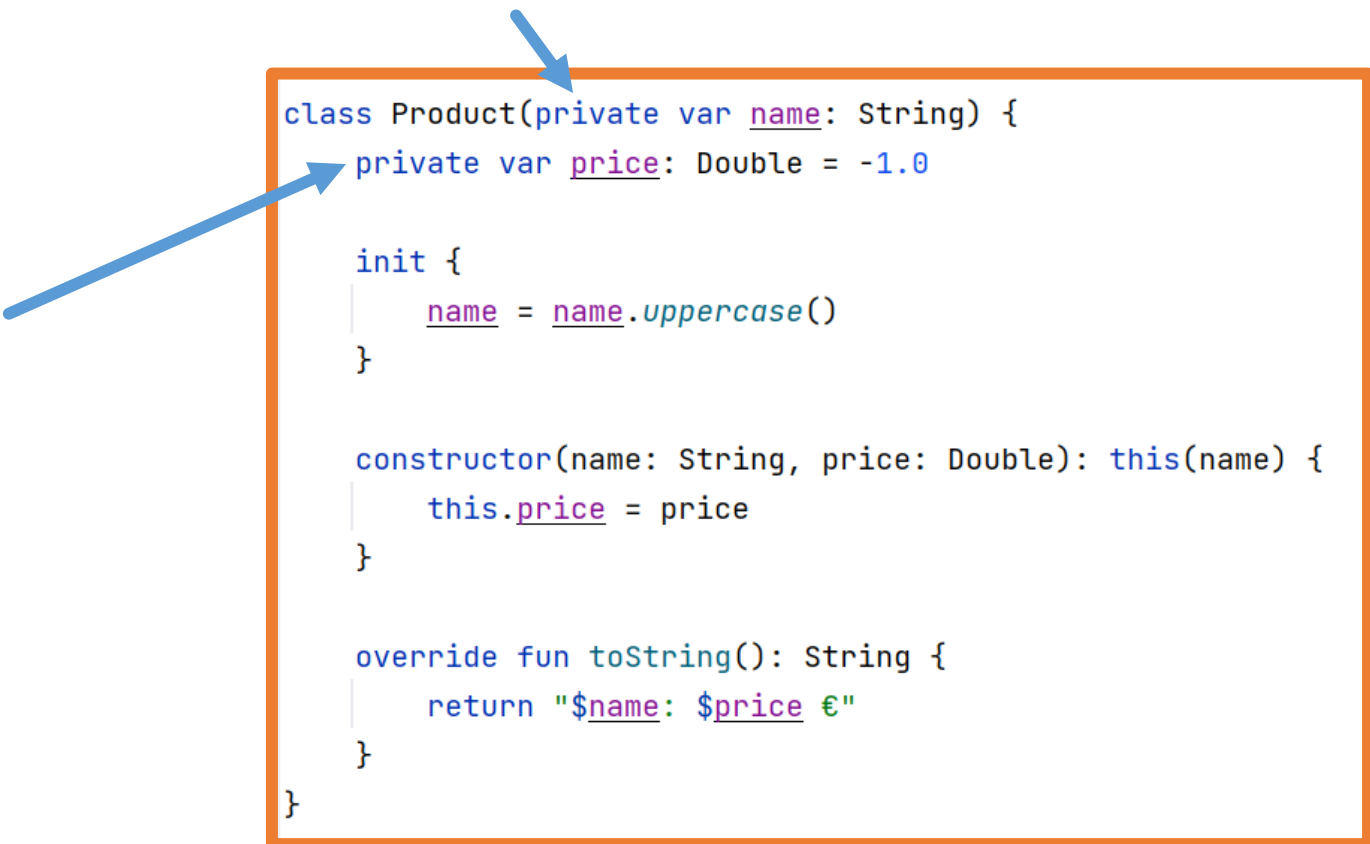
Evidentemente se puede combinar todo lo anterior:

- Constructor primario
- init para realizar acciones en el constructor primario
- Constructor secundario
- getters y setters implícitos
- Sobrecarga de funciones heredadas

```
class Product(var name: String) {  
    var price: Double = -1.0  
  
    init {  
        name = name.uppercase()  
    }  
  
    constructor(name: String, price: Double): this(name) {  
        this.price = price  
    }  
  
    override fun toString(): String {  
        return "$name: $price €"  
    }  
}
```


1.- Programación Orientada a Objetos

Si se quiere cambiar el tipo de acceso de las propiedades de la clase simplemente se debe indicar delante de su declaración.



```
class Product(private var name: String) {  
    private var price: Double = -1.0  
  
    init {  
        name = name.uppercase()  
    }  
  
    constructor(name: String, price: Double): this(name) {  
        this.price = price  
    }  
  
    override fun toString(): String {  
        return "$name: $price €"  
    }  
}
```

1.- Programación Orientada a Objetos

Al ser implícitos, **los getters y los setters tendrán el mismo tipo de acceso que se defina para las propiedades**, y esto **no se puede cambiar**.

Este funcionamiento puede ser un problema ya que si se quiere dotar a las clases de la encapsulación (característica típica de la POO) no se tendrá acceso a las propiedades cuando se usen los objetos de la clase.

```
class Product(private var name: String) {  
    private var price: Double = -1.0  
  
    init {  
        name = name.uppercase()  
    }  
  
    constructor(name: String, price: Double): this(name) {  
        this.price = price  
    }  
  
    override fun toString(): String {  
        return "$name: $price €"  
    }  
}
```

```
val product = Product(name: "Test")  
product.name = "PS5"  
product.price = 495.5
```

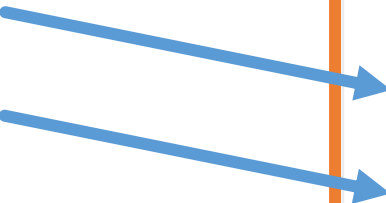


1.- Programación Orientada a Objetos

Para solucionar este problema se pueden crear métodos públicos para acceder a las propiedades privadas.

Aunque esta solución va en contra de la filosofía de Kotlin de ahorrar código.

```
class Product(private var name: String) {  
    private var price: Double = -1.0  
  
    init {  
        name = name.uppercase()  
    }  
  
    constructor(name: String, price: Double): this(name) {  
        this.price = price  
    }  
  
    fun getName(): String {  
        return name  
    }  
  
    fun setName(name: String) {  
        this.name = name  
    }  
  
    override fun toString(): String {  
        return "$name: $price €"  
    }  
}
```



1.- Programación Orientada a Objetos

Si una propiedad es pública se pueden definir el get y el set de manera privada para así proteger el acceso.

En el ejemplo se puede ver que el get seguirá público pero el set no.

Si se necesita cambiar el valor de la propiedad se necesitará definir una función para ello como en la página anterior setName y setPrice.

```
class Product(name: String, price: Double) {  
    var name: String = name  
    |    private set  
    var price: Double = price  
    |    private set  
  
    override fun toString(): String {  
    |    return "$name: $price €"  
    }  
}
```

1.- Programación Orientada a Objetos

Como ya se ha visto anteriormente, una vez declarada una clase su uso es igual que en Java.

El acceso a las propiedades y las funciones se realiza mediante la notación de punto:

```
val product = Product( name: "PS5", price: 495.50)
println("${product.name}: ${product.finalPrice( average: 10.0)} €")
```

2.- Enum class en Kotlin

Las **enum class** permiten definir un conjunto de constantes relacionadas entre sí.

Es una manera segura de disponer de los valores sin temor a errores.

```
enum class Direction {  
    NORTH, SOUTH, WEST, EAST  
}  
  
enum class Color(val rgb: Int) {  
    RED( rgb: 0xFF0000),  
    GREEN( rgb: 0x00FF00),  
    BLUE( rgb: 0x0000FF)  
}
```

```
val direction = Direction.EAST  
println(direction)           // EAST  
println(direction.name)      // EAST  
println(direction.ordinal)    // 3 (posición)  
  
val color = Color.RED  
println(color)                // RED  
println(color.name)           // RED  
println(color.rgb)            // 16711680 (en decimal)  
println(color.rgb.toHexString()) // 00ff0000 (en hexadecimal)  
println(color.ordinal)        // 0 (posición)
```

3.- Data class en Kotlin

Las **data class** son una manera de modelar los datos y la finalidad de sus objetos es simplemente almacenar datos.

Son clases muy simples que solo contienen atributos.

Se utilizan para interactuar con otras clases o con API's externas.

Para declarar una clase de este tipo se usa la palabra **data**.

```
data class Manga(var name: String, var type: String, var volumes: Int)
```

3.- Data class en Kotlin

Kotlin provee a las data class de una serie de **funciones de utilizad**:

- `equals()`: permite comparar dos objetos de la clase.
- `hashCode()`: código hash (este código se usa en la función anterior).
- `copy()`: permite realizar una copia del objeto.
- `toString()`: genera un string legible con los datos del objeto.
- `component1()`, `component2()`...: get a cada propiedad del objeto en su orden de declaración.

3.- Data class en Kotlin

```
data class Manga(var name: String, var type: String, var volumes: Int)
```

```
val dragonBall = Manga(name: "Dragon Ball", type: "Shonen", volumes: 42)
val deathNote = Manga(name: "Death Note", type: "Shonen", volumes: 13)
val naruto = Manga(name: "Naruto", type: "Shonen", volumes: 72)

println(dragonBall)
println(deathNote)
if (dragonBall.equals(naruto)) println("Son el mismo manga.")
else println("Son mangas diferentes.")
```

Salida:

```
Manga(name=Dragon Ball, type=Shonen, volumes=42)
Manga(name=Death Note, type=Shonen, volumes=13)
Son mangas diferentes.
```

3.- Data class en Kotlin

Las propiedades que no aparezcan en el constructor primario no se beneficiarán de las características de las data class.

Por ejemplo, al usar toString solo aparecerán las propiedades que estén en el constructor primario.

Mediante la función copy se pueden copiar objetos completos y durante la copia se pueden cambiar valores de las propiedades.

```
var db = dragonBall.copy()  
var dbz = dragonBall.copy(nombre="DBZ")
```

3.- Data class en Kotlin

Los objetos de una data class se pueden **deconstruir** de manera que se puede extraer el valor de sus propiedades a variables.

Si hay algún valor que no se quiera se debe poner el carácter `_` en su lugar.

Si los valores que no se quieren están al final de la lista basta con no ponerlos.

```
val dragonBall = Manga( name: "Dragon Ball", type: "Shonen", volumes: 42)
val (name, type, volumes) = dragonBall
println("Manga: $name ($type)")
println("Tomos: $volumes")
```

Salida:

```
Manga: Dragon Ball (Shonen)
Tomos: 42
```

```
val (name) = dragonBall
```

```
val (name, type) = dragonBall
```

```
val (name, _, volumes) = dragonBall
```

4.- Funciones de extensión

Kotlin permite **extender la funcionalidad** de las clases existentes ya sean del sistema o propias sin uso de la herencia incluso si la clase existente es final.

Las funciones de extensión se definen fuera de la definición de la clase por ello **solo se podrán utilizar en el ámbito en el que se definan**.

Aunque la función de extensión **se define fuera de la clase** es como si se hubiera **definido desde dentro** comportándose como un método más de la clase, pudiendo hacer uso tanto de las propiedades y los métodos si los hay aunque sean privados.

En el apartado de las funciones ya se vio un ejemplo:

```
// Se añade una función a la clase Int  
fun Int.isOdd(): Boolean {  
    return this % 2 != 0  
}
```

4.- Funciones de extensión

Estas funciones se definen como otras funciones pero usando el nombre de la clase que se quiere extender.

Función de extensión de una clase existente:

```
fun Int.isOdd(): Boolean {  
    return this % 2 != 0  
}
```



```
fun Int.isOdd() = this % 2 != 0
```

Función de extensión de una clase propia

```
class Product(var name: String, var price: Double)  
  
fun Product.sale(salePrice: Double): Double {  
    return if (price > salePrice) salePrice else price  
}
```

4.- Funciones de extensión

Un uso típico de las funciones de extensión se da con las colecciones.

Gracias a una función de extensión se puede realizar una acción sobre cada uno de los elementos de la colección.

```
fun main() {  
    val numbers = mutableListOf(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)  
    println(numbers.addition(quantity: 2))  
}  
  
fun MutableList<Int>.addition(quantity: Int): MutableList<Int> {  
    val numbers = mutableListOf<Int>()  
    for (i in this) numbers.add(i + quantity)  
    return numbers  
}
```

Salida:

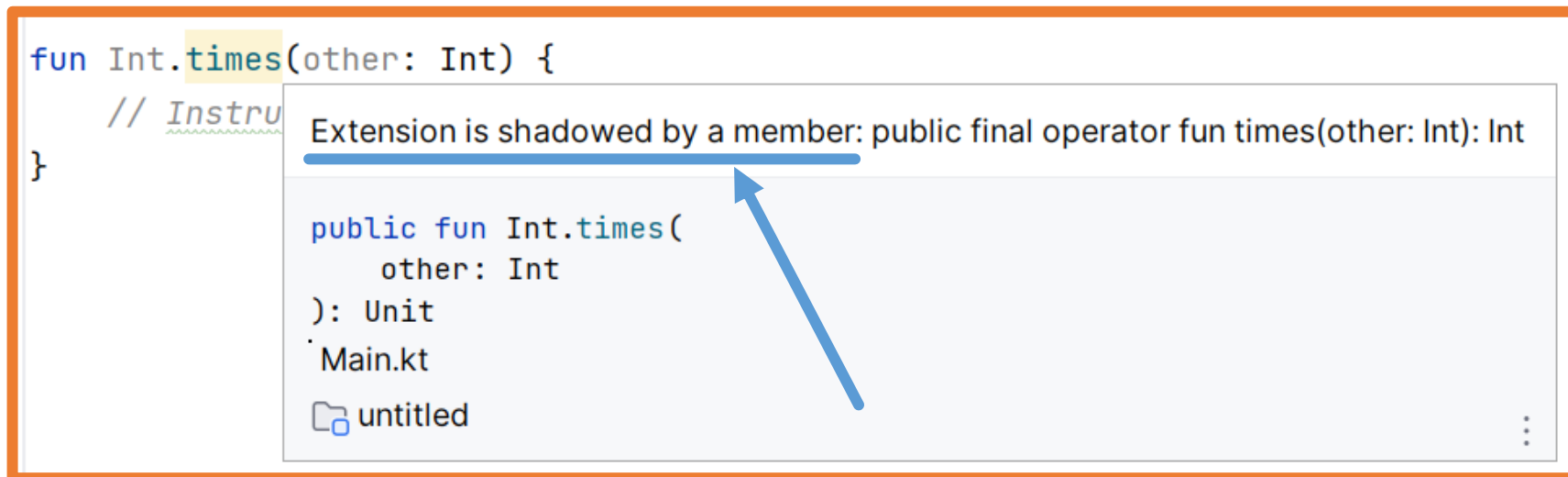
```
[3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
```

4.- Funciones de extensión

Una función de extensión **no puede sobre escribir a una función miembro.**

Una función de extensión **sí que puede sobrecargar a una función miembro.**

Aunque no se muestre un error, si se intenta sobrescribir una función miembro mediante una función de extensión, en ejecución siempre se llamará a la función miembro.



```
fun Int.times(other: Int) {  
    // Instru  
}
```

Extension is shadowed by a member: public final operator fun times(other: Int): Int

```
public fun Int.times(  
    other: Int  
): Unit  
Main.kt  
untitled
```

The screenshot shows a code editor with a Kotlin extension function `fun Int.times(other: Int) {` and a comment `// Instru`. A warning message is displayed: `Extension is shadowed by a member: public final operator fun times(other: Int): Int`. Below the warning, the member function `public fun Int.times(other: Int): Unit` is shown. A blue arrow points from the warning message to the member function. The editor also shows `Main.kt` and `untitled` in the sidebar.

5.- Singleton

En Kotlin se pueden crear **singleton** que son **objetos únicos en su clase**, esto significa que no podrá haber más instancias de esa clase.

```
object Author {  
    var name = "Álex Torres"  
    var company = "2 DAM"  
    var date = Date()  
  
    override fun toString():String {  
        return """"$name ($company)  
                |$date""".trimMargin()  
    }  
}
```

Se le pueden añadir funciones de extensión como con cualquier otra clase.

Si se declaran de manera global en el archivo .kt se podrán añadir funciones de extensión como con cualquier otra clase.

Práctica

Actividad

Clase de película

Actividad

Dados

6.- Herencia

En **Kotlin por defecto todas las clases son finales**, lo cual significa que no puede haber herencia si no se indica explícitamente.

Para indicar que una clase puede ser súper clase se debe usar la palabra reservada **open**.

Si se usa el constructor primario y los getters y setters por defecto:

```
open class Product(var name: String, var price: Double)

class Monitor(name: String, price: Double): Product(name, price)
```

Con este código se tiene:

- La clase Product con el constructor, los getter y los setters
- La clase Monitor con su constructor que llama al constructor de la clase padre, y hereda los getter y los setters de la clase padre.

6.- Herencia

Se pueden añadir propiedades y métodos nuevos.

```
open class Product(var name: String, var price: Double)

class Monitor(name: String, price: Double): Product(name, price) {
    var size: Int = 0

    constructor(name: String, price: Double, size: Int): this(name, price)

    override fun toString(): String {
        return "$name ($size\"): $price €"
    }
}
```

6.- Herencia

```
open class Product(var name: String, var price: Double)

class Monitor(name: String, price: Double): Product(name, price) {
    var size: Int = 0

    constructor(name: String, price: Double, size: Int): this(name, price)

    override fun toString(): String {
        return "$name ($size\"): $price €"
    }
}
```

En la clase hija automáticamente se crea **el constructor primario que llamará al constructor de la clase madre.**

Además, se ha creado un constructor secundario que mediante **this** llamará al constructor primario.

7.- Clases abstractas

Para declarar una clase abstracta se usa la palabra **abstract** delante de la palabra **class**.

De una clase abstracta no se pueden instanciar objetos.

```
abstract class Product(var name: String, var trademark: String) {
    abstract var price: Double
    abstract fun powerOn()
    abstract fun powerOff()
    fun information() {
        println("$name ($trademark): $price €")
    }
}

class Monitor(name: String, trademark: String, var color: String, override var price: Double) : Product(name, trademark) {
    override fun powerOn() {
        println("Encendido")
    }

    override fun powerOff() {
        println("Apagado")
    }
}
```



UD2.4 – Fundamentos de Kotlin

2º CFGS
Desarrollo de Aplicaciones Multiplataforma
2023-24

1.- Funciones de alcance

Kotlin ofrece las llamadas **Scope functions** que permiten ejecutar un bloque de código **en el contexto del objeto que las llama**.

Al ejecutarse en el contexto del objeto que las llama, dentro del cuerpo de la función está disponible dicho objeto.

Existen cinco funciones de alcance:

- let
- run
- with
- apply
- also

1.- Funciones de alcance

Función	Uso	Contexto	Devuelve
let	Ejecutar bloque de código asegurándose de que el objeto no es null.	Propio objeto accesible con it	Resultado de la última instrucción
apply	Configuración de un objeto.	Propio objeto accesible con this	Propio objeto
run	Configuración de un objeto y ejecución de instrucciones sobre él.	Propio objeto accesible con this	Resultado de la última instrucción
run	Ejecutar instrucciones cuando se requiere una expresión, se usa sin que sea llamada desde un objeto.	-	Resultado de la última instrucción
also	Seguir realizando instrucciones sobre el objeto.	Propio objeto accesible con it	Propio objeto
with	Agrupar llamadas a funciones de un objeto.	Propio objeto accesible con this	Resultado de la última instrucción

Estas funciones devuelven un valor pero no es necesario capturarlo (guardarlo) en ninguna variable.

1.- Funciones de alcance

Todas las funciones de alcance se utilizan con un bloque de llaves { } (función lambda).

IntelliJ IDEA muestra las pistas en el código (hints) para ayudar a entender cómo funciona cada una de ellas.

```
class Product(var name: String, var price: Double, var year: Int)

val product = Product( name: "PS5", price: 495.50, year: 2021)
var result = product.let { it: Product
    println("Se va a aplicar un descuento")

    it.price * 0.85 ^let
}
```

Contexto

^let: la flecha indica que se devuelve el resultado

```
class Product(var name: String, var price: Double, var year: Int)

val product = Product( name: "PS5", price: 495.50, year: 2021)
var result = product.let { it: Product
    println("Si el producto es antiguo se aplicará un descuento")
    if (it.year < 2023) it.price * 0.85 ^let
    else it.price ^let
}
```

Contexto

^let: como hay 2 caminos posibles aparece 2 veces

2.- let

let

Contexto: el objeto desde el cual se llama, accesible con **it**.

Devuelve: el resultado de la última instrucción.

```
class Product(var name: String, var price: Double, var year: Int)

val product: Product = Product(name: "PS5", price: 495.50, year: 2021)
var result = product.let { it: Product
    println("Si el producto es antiguo se aplica un descuento")
    if (it.year < 2023) it.price * 0.85 ^let
    else it.price ^let
}
```

2.- let

let

El uso de let asegura que el objeto no será null antes de ejecutar las instrucciones.

```
class Product(var name: String, var price: Double, var year: Int)

var product: Product? = Product(name: "PS5", price: 495.50, year: 2021)
val number = (1..100).random()
if (number % 2 == 0) product = null
var result = product?.let { it: Product
    println("Si el producto es antiguo se aplica un descuento")
    if (it.year < 2023) it.price * 0.85 ^let
    else it.price ^let
}
```

En el caso de que **el objeto producto** sea null el bloque **let** no se ejecutará.

3.- run

run

Tiene dos usos:

- 1.- Llamada desde un objeto como let, pero no controla si la variable es null.

El contexto: el objeto desde el cual se llama, accesible con **this**. No es necesario poner **this**. para acceder a las propiedades.

Devuelve: el resultado de la última instrucción.

```
class Product(var name: String, var price: Double, var year: Int) {  
    fun addPercentageToPrice(percentage: Double) {  
        price += price * percentage / 100  
    }  
}  
  
val product: Product = Product( name: "PS5", price: 495.50, year: 2021)  
val result = product.run { this: Product  
    // this.price /=2  
    name += " [OFERTA]"  
    if (price < 1000.0) addPercentageToPrice( percentage: 12.5)  
    price ^run  
}  
println("Producto: ${product.name} ${product.price}")  
println(result)
```

Salida:

```
Producto: PS5 [OFERTA] 557.4375  
557.4375
```

3.- run

run

Tiene dos usos:

2.- Llamada sin usar objeto.

El contexto: no hay contexto.

Devuelve: el resultado de la última instrucción.

```
val name = "Rick Sanchez"
val containsR = run {
    var letterR = false
    for (letter in name) {
        if (letter == 'R') {
            letterR = true
            break
        }
    }
    letterR ^run
}
```

4.- also

also

Permite realizar acciones extra (also = además) sobre el objeto que la llama.

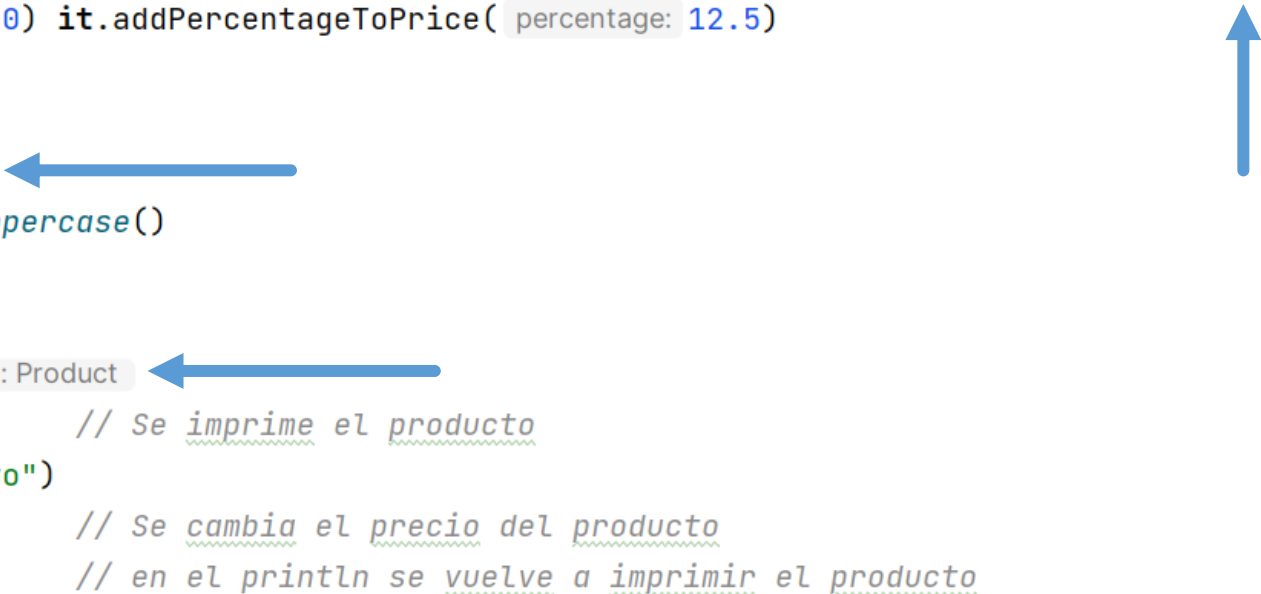
El contexto: el objeto desde el cual se llama, accesible con **it**.

Devuelve: el propio objeto (se hace automáticamente).

```
val product: Product = Product( name: "Nintendo Switch", price: 350.0, year: 2021).also { it: Product
    if (it.price < 1000.0) it.addPercentageToPrice( percentage: 12.5)
}

product.also { it: Product
    it.name = it.name.uppercase()
}

println(product.also { it: Product
    println("---> $it")    // Se imprime el producto
    println("Precio nuevo")
    it.price *= 0.85      // Se cambia el precio del producto
                          // en el println se vuelve a imprimir el producto
})
```



5.- apply

apply

Se utiliza para configurar (asignar valores) a un objeto.

El contexto: el objeto desde el cual se llama, accesible con **this**.

Devuelve: el propio objeto (se hace automáticamente).

```
val product: Product = Product( name: "Nintendo Switch", price: 300.0, year: 2021)
product.apply { this: Product ←
    // this.name = "Switch OLED"
    name = "Switch OLED"
    price = 349.90
}
```

6.- with

with

Permite agrupar acciones sobre un objeto.

El contexto: el objeto desde el cual se llama, accesible con **this**.

Devuelve: el resultado de la última instrucción.

```
val product: Product = Product( name: "Nintendo Switch", price: 300.0, year: 2021)
with(product) { this: Product
    println(
        """$name
        | Precio anterior: $price €
        """.trimMargin()
    )
    if (price < 1000.0) addPercentageToPrice( percentage: 12.5)
    println(" Precio nuevo: $price €")

    println(this)
}
```


6.- with

with

Permite agrupar acciones sobre un objeto.

El contexto: el objeto desde el cual se llama, accesible con **this**.

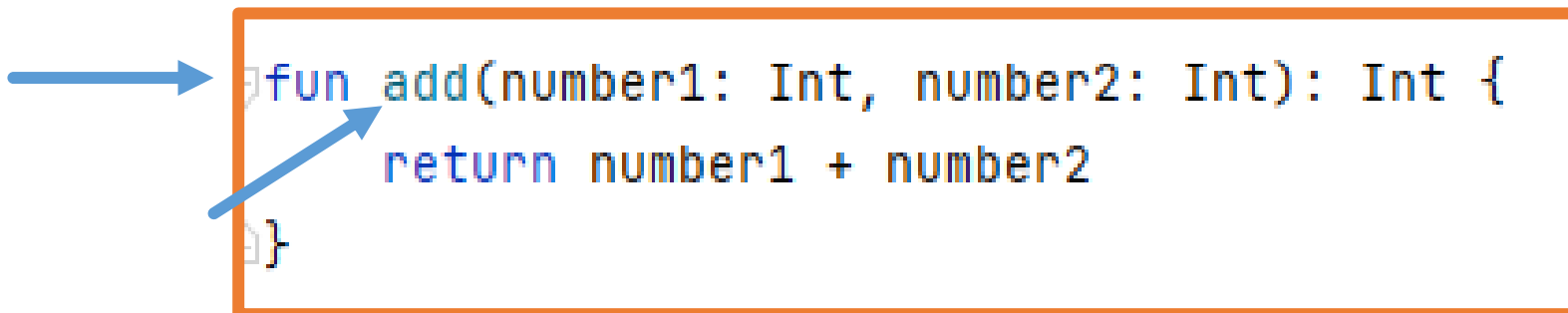
Devuelve: el resultado de la última instrucción.

```
val product: Product = Product( name: "Nintendo Switch", price: 300.0, year: 2021)
val newProduct = with(product) { this: Product ←
    println(
        """$name
        | Precio anterior: $price €
        """.trimMargin()
    )
    if (price < 1000.0) addPercentageToPrice( percentage: 12.5)
    println(" Precio nuevo: $price €")

    println(this)
    this ^with ←
}
```

7.- Funciones lambda

Como norma general una función se debe declarar y se le debe asignar un identificador para poder usarse:



```
fun add(number1: Int, number2: Int): Int {  
    return number1 + number2  
}
```

The diagram illustrates a function declaration in Kotlin. A blue arrow points to the word 'fun', indicating the keyword used to declare a function. Another blue arrow points to the parameter list '(number1: Int, number2: Int)', indicating the inputs to the function. The function body is enclosed in curly braces and contains a return statement 'return number1 + number2'.

Las funciones **lambda**, también llamadas funciones anónimas o funciones flecha son funciones que **no están declaradas (no tienen identificador)** y se **utilizan como una expresión**.

7.- Funciones lambda

Las funciones lambda **se escriben entre llaves { }**, pueden tener parámetros o no y deben tener un cuerpo.

Si no tiene parámetros se deben poner directamente las instrucciones.

El resultado de la última instrucción de una lambda se devolverá, si la función tiene varias instrucciones IntelliJ IDEA lo indica con las pistas en el código (hints).

```
{ println("Desde una lambda") }
```

```
{  
    var text = "Desde una lambda"  
    text += "!"  
    text ^lambda  
}
```

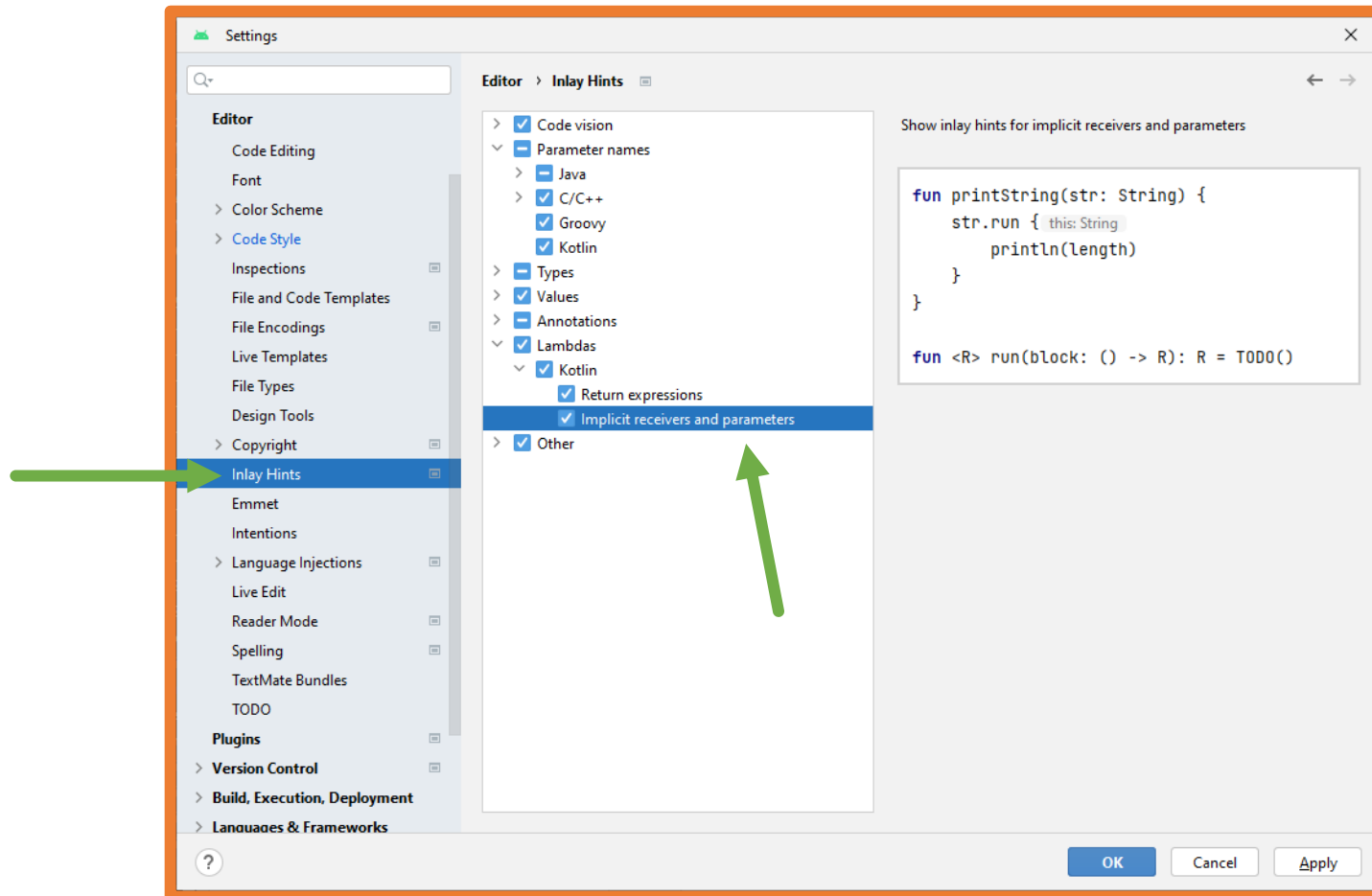


```
{  
    val first10EvenInts: MutableList<Int> = mutableListOf()  
    for (i in 0..until(10)) {  
        first10EvenInts.add(i * 2)  
    }  
    first10EvenInts ^lambda  
}
```



7.- Funciones lambda

Si no aparecen las pistas de código (hints) en IntelliJ IDEA se pueden activar desde la configuración **File → Settings (CTRL+ALT+S)**:



7.- Funciones lambda

Si tiene parámetros, para separarlos del cuerpo se utilizan los caracteres -> de ahí que se les conozca también como funciones flecha.

```
{ name: String -> "Hola $name! Te saludo desde una lambda" }
```

```
{ x: Int, y: Int -> x + y }
```

```
{ init: Int, final: Int ->
  val evenInterval: MutableList<Int> = mutableListOf()
  for (i in init ≤ until < final) {
    if (i % 2 == 0) evenInterval.add(i)
  }
  evenInterval ^lambda
}
```

7.- Funciones lambda

En las lambdas escritas anteriormente es Kotlin quien deduce los tipos de datos.

Se pueden indicar explícitamente los tipos de datos:

```
(String) -> String = { name: String ->
    "Hola $name! Te saludo desde una lambda"
}
```

```
(Int, Int) -> MutableList<Int> = { init: Int, final: Int ->
    val evenInterval: MutableList<Int> = mutableListOf()
    for (i in init..until<Int>(final)) {
        if (i % 2 == 0) evenInterval.add(i)
    }
    evenInterval ^lambda
}
```

En este caso si la lambda solo tiene un parámetro se puede omitir y utilizar **it**:

```
(String) -> String = { it: String
    "Hola $it! Te saludo desde una lambda"
}
```

7.- Funciones lambda

Las funciones lambda se utilizan como una expresión, esto significa que deben de estar asignadas a algún elemento.

Si se asigna una lambda a una variable su uso es el mismo que con una función normal.

Esto no aporta ninguna funcionalidad nueva.

```
val greeting = { name: String -> "Hola $name! Te saludo desde una lambda" }  
println(greeting("Rick"))
```

```
val add: (Int, Int) -> Int = { number1: Int, number2: Int -> number1 + number2 }  
println(add(3,48))
```

```
val evenInterval = { init: Int, final: Int ->  
    val evenInterval: MutableList<Int> = mutableListOf()  
    for (i in init until final) {  
        if (i % 2 == 0) evenInterval.add(i)  
    }  
    evenInterval ^lambda  
}  
println(evenInterval(0, 25))
```

7.- Funciones lambda

Las funciones lambda **se pueden pasar como parámetros a otras funciones**.

Esto sí que aporta una funcionalidad no vista hasta ahora y es una práctica muy extendida actualmente en muchos lenguajes de programación y frameworks.

Gracias al paso de lambdas como parámetro en funciones se puede:

- Crear funciones callback.
- Devolver diferentes respuestas desde una función.
- Ofrecer al programador que introduzca su propia lógica.

7.- Funciones lambda

Sintaxis de paso de función lambda como parámetro de una función:

```
fun nameOfFun(parameter1: Type, lambdaName: (Type1, Type2...) -> TypeReturn) { ... }
```

En el caso de que la lambda no reciba parámetros:

```
fun nameOfFun(parameter1: Type, lambdaName: () -> TypeReturn) { ... }
```

En el caso de que la lambda no devuelva nada:

```
fun nameOfFun(parameter1: Type, lambdaName: (Type1, Type2...) -> Unit) { ... }
```

En el caso de que la lambda no reciba parámetros ni devuelva nada:

```
fun nameOfFun(parameter1: Type, lambdaName: () -> Unit) { ... }
```

7.- Funciones lambda

Crear funciones callback

Las funciones callback permiten asegurarse que un conjunto de instrucciones se ejecuta después de una instrucción concreta. Su uso es muy típico cuando existe ejecución asíncrona de instrucciones, hilos...

Las siguientes funciones sobrecargadas **reciben una función lambda** que se ejecuta al final de todo el cuerpo de la función **doLogin**:

```
fun doLogin(username: String, pass: String, lambda: (String, String) -> Unit) {  
    // Instrucciones que conectan a la BBDD y se realiza el login  
    Thread.sleep(5000)  
    lambda(username, pass)  
}  
  
fun doLoginGuest(lambda: () -> Unit) {  
    // Instrucciones que conectan a la BBDD y se realiza el login invitado  
    Thread.sleep(5000)  
    lambda()  
}
```

7.- Funciones lambda

Crear funciones callback

Ejemplos de llamadas a la función **doLogin**.

```
fun doLogin(username: String, pass: String, lambda: (String, String) -> Unit) {  
    // Instrucciones que conectan a la BBDD y se realiza el login  
    Thread.sleep(5000)  
    lambda(username, pass)  
}  
  
fun doLoginGuest(lambda: () -> Unit) {  
    // Instrucciones que conectan a la BBDD y se realiza el login invitado  
    Thread.sleep(5000)  
    lambda()  
}
```

```
doLogin( username: "Rick", pass: "c-137", { username, pass -> println("Login: $username - $pass") })
```

Si la función lambda es el último parámetro se puede sacar fuera de los paréntesis:

```
doLogin( username: "Morty", pass: "Sm1th") { username, pass ->  
    println("Login: $username - $pass")  
}
```

Si la función lambda es el único parámetro se pueden quitar los paréntesis de la llamada:

```
doLoginGuest {  
    println("Login invitado")  
}
```

7.- Funciones lambda

Devolver diferentes respuestas desde una función

A una función se le pueden pasar tantos parámetros como se quiera y todos/varios de esos parámetros pueden ser una función lambda.

De esta manera se puede ejecutar una u otra función lambda según se necesite.

```
fun calculate(  
    number1: Double,  
    number2: Double,  
    operation: Char  
) : Double {  
    when (operation) {  
        '+' -> return number1 + number2  
        '-' -> return number1 - number2  
        '*' -> return number1 * number2  
        '/' -> return number1 / number2  
        else -> return 0.0  
    }  
}
```

```
fun calculate(  
    number1: Double,  
    number2: Double,  
    operation: Char,  
    correct: (result: Double) -> Unit,  
    error: (error: String) -> Unit  
) {  
    if (number1 < 0) {  
        error( error: "El primer número es menor que cero.")  
    } else if (number2 < 0) {  
        error( error: "El segundo número es menor que cero.")  
    } else {  
        when (operation) {  
            '+' -> correct( result: number1 + number2)  
            '-' -> correct( result: number1 - number2)  
            '*' -> correct( result: number1 * number2)  
            '/' -> {  
                if (number2 == 0.0) error( error: "No se puede dividir entre cero.")  
                else correct( result: number1 / number2)  
            }  
            else -> error( error: "Operación no válida")  
        }  
    }  
}
```

7.- Funciones lambda

Devolver diferentes respuestas desde una función

```
fun calculate(
    number1: Double,
    number2: Double,
    operation: Char,
    correct: (result: Double) -> Unit,
    error: (error: String) -> Unit
) {
    if (number1 < 0) {
        error( error: "El primer número es menor que cero.")
    } else if (number2 < 0) {
        error( error: "El segundo número es menor que cero.")
    } else {
        when (operation) {
            '+' -> correct( result: number1 + number2)
            '-' -> correct( result: number1 - number2)
            '*' -> correct( result: number1 * number2)
            '/' -> {
                if (number2 == 0.0) error( error: "No se puede dividir entre cero.")
                else correct( result: number1 / number2)
            }
            else -> error( error: "Operación no válida")
        }
    }
}
```

Uso:

```
var result = ""
calculate(
    number1 = 9.0,
    number2 = 6.0,
    operation = '-',
    correct = { it: Double
        result = it.toString()
    },
    error = { it: String
        result = it
    }
)
println("El resultado es: $result")
```

Salida:

El resultado es: 3.0

7.- Funciones lambda

Devolver diferentes respuestas desde una función

Dependiendo del cuerpo de la función lambda se podrán ejecutar una o varias de estas funciones.

En el ejemplo, tanto en la función **correct** como en la función **error** se guarda el resultado obtenido devuelto por lo que solo tendrá efecto última llamada a las funciones lambda sea cual sea:

```
fun calculate(
    number1: Double,
    number2: Double,
    operation: Char,
    correct: (result: Double) -> Unit,
    error: (error: String) -> Unit
) {
    if (number1 < 0) {
        error( error: "El primer número es menor que cero.")
    } else if (number2 < 0) {
        error( error: "El segundo número es menor que cero.")
    } else {
        error( error: "Sin error")
        correct( result: 0.0)
        when (operation) {
            '+' -> correct( result: number1 + number2)
            '-' -> correct( result: number1 - number2)
            '*' -> correct( result: number1 * number2)
            '/' -> {
                if (number2 == 0.0) error( error: "No se puede dividir entre cero.")
                else correct( result: number1 / number2)
            }
            else -> error( error: "Operación no válida")
        }
    }
}
```

Uso:

```
var result = ""
calculate(
    number1 = 9.0,
    number2 = 6.0,
    operation = '-',
    correct = { it: Double
        result = it.toString()
    },
    error = { it: String
        result = it
    }
)
println("El resultado es: $result")
```

Salida:

El resultado es: 3.0

7.- Funciones lambda

Devolver diferentes respuestas desde una función

Dependiendo del cuerpo de la función lambda se podrán ejecutar una o varias de estas funciones.

Ahora se ha cambiado el cuerpo de las funciones **correct** y **error** para que muestren el resultado obtenido:

```
fun calculate(
    number1: Double,
    number2: Double,
    operation: Char,
    correct: (result: Double) -> Unit,
    error: (error: String) -> Unit
) {
    if (number1 < 0) {
        error( error: "El primer número es menor que cero.")
    } else if (number2 < 0) {
        error( error: "El segundo número es menor que cero.")
    } else {
        error( error: "Sin error")
        correct( result: 0.0)
        when (operation) {
            '+' -> correct( result: number1 + number2)
            '-' -> correct( result: number1 - number2)
            '*' -> correct( result: number1 * number2)
            '/' -> {
                if (number2 == 0.0) error( error: "No se puede dividir entre cero.")
                else correct( result: number1 / number2)
            }
            else -> error( error: "Operación no válida")
        }
    }
}
```

Uso:

```
calculate(
    number1 = 9.0,
    number2 = 6.0,
    operation = '-',
    correct = { it: Double
        println("El resultado es: $it")
    },
    error = { it: String
        println("Error: $it")
    }
)
```

Salida:

```
Error: Sin error
El resultado es: 0.0
El resultado es: 3.0
```

7.- Funciones lambda

Devolver diferentes respuestas desde una función

Si se quiere **definir un parámetro tipo función lambda como opcional** se debe indicar con los caracteres = {}.

```
fun calculate(  
    number1: Double,  
    number2: Double,  
    operation: Char,  
    correct: (result: Double) -> Unit,  
    error: (error: String) -> Unit = {}  
) {  
    if (number1 < 0) {  
        error( error: "El primer número es menor que cero.")  
    } else if (number2 < 0) {  
        error( error: "El segundo número es menor que cero.")  
    } else {  
        when (operation) {  
            '+' -> correct( result: number1 + number2)  
            '-' -> correct( result: number1 - number2)  
            '*' -> correct( result: number1 * number2)  
            '/' -> {  
                if (number2 == 0.0) error( error: "No se puede dividir entre cero.")  
                else correct( result: number1 / number2)  
            }  
        }  
        else -> error( error: "Operación no válida")  
    }  
}
```

Uso:

```
calculate(  
    number1 = 9.0,  
    number2 = 6.0,  
    operation = 'o',  
    correct = { it: Double  
        println("El resultado es: $it")  
    },  
    error = { it: String  
        println("Error: $it")  
    }  
)
```

Salida:

Error: Operación no válida

Uso:

```
calculate(  
    number1 = 9.0,  
    number2 = 6.0,  
    operation = 'o',  
    correct = { it: Double  
        println("El resultado es: $it")  
    },  
)
```

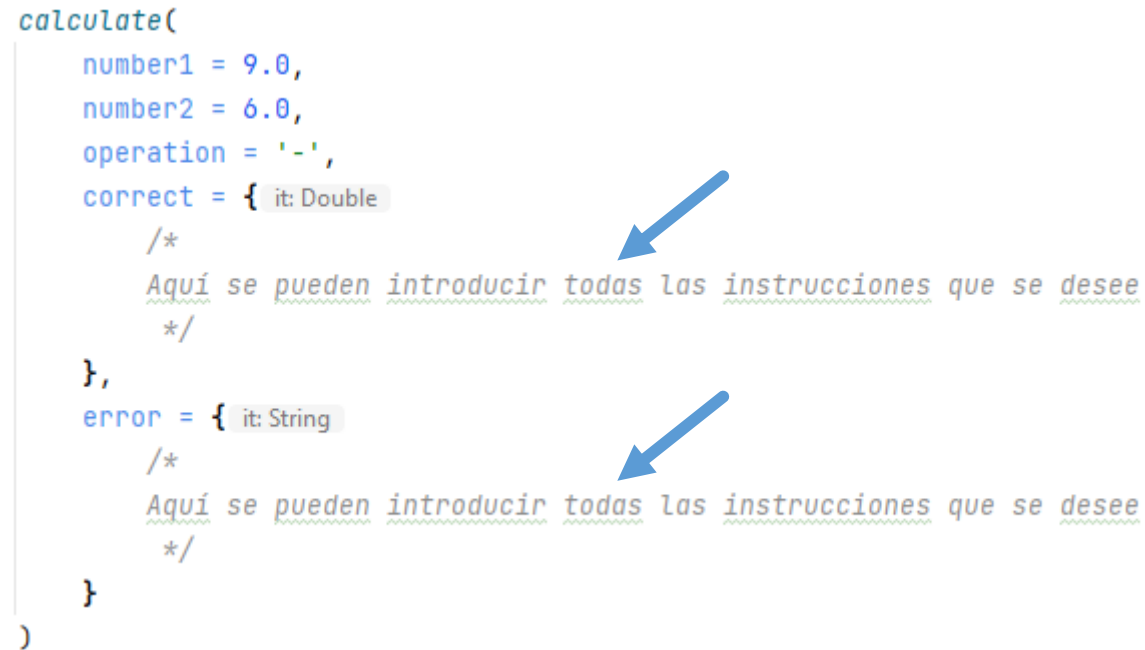
Salida:

7.- Funciones lambda

Ofrecer al programador que introduzca su propia lógica

Si la función **calculate** estuviera en una librería externa al usarla se da la opción al programador de incorporar todas las instrucciones que quiera.

```
calculate(  
    number1 = 9.0,  
    number2 = 6.0,  
    operation = '-',  
    correct = { it: Double  
        /*  
        Aquí se pueden introducir todas las instrucciones que se desee  
        */  
    },  
    error = { it: String  
        /*  
        Aquí se pueden introducir todas las instrucciones que se desee  
        */  
    }  
)
```

The diagram shows a code snippet for a function named 'calculate'. The function has four parameters: 'number1' (9.0), 'number2' (6.0), 'operation' ('-'), and a block parameter 'correct' of type 'Double'. The 'correct' block contains a comment indicating where instructions can be added. A blue arrow points from the text 'Aquí se pueden introducir todas las instrucciones que se desee' to the 'correct' block. Another blue arrow points from the same text to the 'error' block, which is also a block parameter of type 'String' and contains a similar comment.

7.- Funciones lambda

Anteriormente ya se ha hecho uso de funciones lambda como parámetros de otras funciones, exactamente como último parámetro de la función:

- En la creación de arrays:

```
var reverse = Array( size: 50) { 50-it }
```



- En las funciones forEach, filter y map:

```
val numbers = listOf(1, 2, 3, 4, 5, 6)  
numbers.forEach { it: Int  
    print("${it*-1} ")  
}
```



- En las funciones de alcance:

```
val product: Product = Product( name: "Nintendo Switch", price: 300.0, year: 2021)  
product.apply { this: Product  
    // this.name = "Switch OLED"  
    name = "Switch OLED"  
    price = 349.90  
}
```



7.- Funciones lambda

El uso de funciones lambda es muy importante en el desarrollo de aplicaciones móviles Android.

Sobre todo al usar **Jetpack Compose** como se verá en el tema siguiente.

```
MyMoviesApp {  
    Scaffold(topBar = { MainAppBar(mediaItem.title) }) { it: PaddingValues  
        Row(  
            modifier = Modifier.padding()  
        ) { this: RowScope  
            MainContent(  
                onNavigate = { it: MediaItem  
                    currentItem = it.id  
                },  
                modifier = Modifier.weight(1f)  
            )  
            DetailContent(  
                mediaItem = mediaItem,  
                modifier = Modifier.weight(2f)  
            )  
        }  
    }  
}
```

Lambdas

Práctica

Actividad 6

Electrodomésticos